Indie Devs Documentation

Alejandro Garcia Jr
Garrett DeYong
Nicholas Nolen
William Waweru
Gan-Orgil Gantumur

October 28, 2026

Change Logs

Iteration I

- **Documentation Changes:** Defined the project overview, functional/non-functional requirements, user stories, and identified key technical risks.
- **Functional Requirements:** Authored 15 fully-dressed use cases covering core workflows for Fans, Artists, and Admins.
- **Design Changes:** Developed the conceptual system Domain Model, Use Case Diagram, and initial UI Mockups for core screens.

Iteration II

- **Functional & Design Changes:** Designed the backend architecture, objects, and logic for the features that were defined and assigned in Iteration I.
- **Documentation (Sequence Diagrams):** Built the Design Class Diagram. Created detailed Sequence Diagrams for all team member use cases, mapping interactions between actors, Controllers, Services, and Repositories.
- **Quality Assurance:** Defined and documented automated backend test cases for Iteration 2 scenarios mapping to standard and alternative use case flows.

Contents

Change Logs **1**

1 Introduction **4**

2 Project Overview **5**
2.1 Product Perspective 5
2.2 Assumptions and Dependencies 5

3 Requirements Analysis **6**

4 User Story **7**

5 Key Technical Challenges **9**

6 What's New? **10**

7 Use Cases **11**
7.1 Alejandro Garcia Jr – Use Cases 11
7.1.1 UC: Search for Artist – Fan 11
7.1.2 UC: Review Report – Admin 12
7.1.3 UC: Create Event – Artist 13
7.2 Garrett DeYong – Use Cases 14
7.2.1 UC: Make a Post – Artist 14
7.2.2 UC: Message Another User (Beginning a New Conversation) 15
7.2.3 UC: Monitor Comments – Artist 16
7.3 Nicholas Nolen – Use Cases 17
7.3.1 UC: Update Event 17
7.3.2 UC: Unban Fan 18
7.3.3 UC: Recieve Reccomendation 19
7.4 Gan-Orgil Gantumur – Use Cases 19
7.4.1 UC: Suspending Artist Account – Admin 19
7.4.2 UC: Comment on Artwork – Fan 20
7.4.3 UC: Approve Artist Verification Application – Admin 21
7.5 William Waweru – Use Cases 21
7.5.1 UC: Follow Artist – Fan 21
7.5.2 UC: Manage Fan Profile – Fan 22
7.5.3 UC: Report Inappropriate Content – Fan 23

8 Sequence Diagrams **25**
8.1 Garrett DeYong – Sequence Diagrams . . . 25
8.2 Nicholas Nolen – Sequence Diagrams . . . 27
8.3 Gan-Orgil Gantumur – Sequence Diagrams . . . 30
8.4 Alex Garcia – Sequence Diagrams . . . 32
8.5 William Waweru – Sequence Diagrams . . . 35
8.5.1 Follow Artist – Fan . . . 35
8.5.2 Manage Fan Profile – Fan . . . 35
8.5.3 Report Inappropriate Content – Fan . . . 36

9 Use Case Diagram **37**

10 Domain Model **38**

11 Design Class Diagram **39**
11.1 Design Classes – Search for Artist, Create Event, Review Report . . . 40

12 Implementation Overview **43**
12.1 Architecture . . . 43
12.2 Key Design Decisions . . . 43

13 Operation Contracts **45**

14 Testing **47**
14.1 JUnit Testing Summary . . . 47
14.2 Test Cases Table . . . 47

15 Setup / Installation Guide **77**
15.1 Required Tools . . . 77
15.2 Environment Configuration . . . 77
15.3 Download the Project . . . 78
15.4 Start the Backend . . . 78
15.5 Install and Start the Frontend . . . 79
15.6 Run the Automated Checks . . . 79
15.6.1 Backend JUnit Tests . . . 79
15.6.2 Frontend Checks . . . 79
15.6.3 Browser Integration Tests . . . 80
15.7 Troubleshooting and Resetting Sample Data . . . 80

16 UI Mockups **82**
16.1 Make a Post – Artist . . . 82
16.2 Search for Artist – Fan . . . 84
16.3 Create Event – Artist . . . 85

1. Introduction

ArtistConnect is a web-based platform designed to help independent artists share their creative work and build meaningful relationships with their audiences. Artists often rely on several separate services to publish media, announce events, and communicate with supporters. Fans, meanwhile, may find it difficult to discover emerging artists in their own communities.

ArtistConnect brings these activities together in one place. Artists can maintain profiles, publish multimedia posts, create events, and interact with fans. Fans can search for artists by name, style, or location, follow creators, and engage with their content. Administrators support a safe community by reviewing reports and managing accounts. The project aims to make independent creative communities easier to discover, participate in, and sustain.

2. Project Overview

ArtistConnect provides a shared online environment for independent artists, fans, and platform administrators. Its core capabilities include artist discovery, multimedia publishing, event promotion, community interaction, direct messaging, and content moderation. Rather than focusing on a single medium, the platform accommodates different kinds of creative work and emphasizes connections between artists and their local audiences.

2.1 Product Perspective

ArtistConnect is intended to operate as a standalone web application accessible through a browser. The user interface provides distinct workflows for artists, fans, and administrators. The envisioned system uses an application layer to handle business rules and a database to store user accounts, artist profiles, posts, events, and relationships between users. The project also anticipates deployment through a cloud hosting service.

The current user-interface previews demonstrate representative workflows, including creating a post, searching for artists, and creating an event. These previews illustrate the intended user experience; browser-local demonstration data should not be mistaken for completed backend persistence, live notifications, or deployed production functionality.

2.2 Assumptions and Dependencies

The project assumes that users can access a modern web browser and that registered accounts will be associated with appropriate roles and permissions. Artist discovery depends on artists providing searchable profile information, such as their name, creative style, and location. Event creation depends on artists supplying sufficient event details, and publication of ticket pricing may require administrative approval.

The full system depends on the availability of backend services, persistent data storage, and hosting infrastructure. Multimedia uploads will also require suitable file validation and storage. Features involving live messaging, notifications, recommendations, or automated moderation may depend on additional services and must be distinguished from functionality demonstrated only in the interface prototype.

3. Requirements Analysis

Customer / Stakeholder Perspective: Small independent artists need a platform to host their art and foster deeper connections with their fan communities. As an artist, I want a hybrid social media and art hosting site to upload my music, images, and stories, promote events, and interact directly with my supporters through messages and videos. As a fan, I want to discover new artists near me, follow their updates, attend their shows, and interact with other members of the community. As an administrator, I need tools to moderate the platform, review applications, and keep the community safe.

Functional Requirements:

- The system must support role-based access control with distinct permissions and workflows for Fans, Artists, and Administrators.
- The system must allow Artists to maintain a profile, upload multimedia content (music, art images, stories, videos), and promote their events.
- The system must enable Fans to maintain an "about" profile and discover new artists using a proximity-based radius search.
- The system must provide a messaging infrastructure allowing fans to message artists and each other.
- The system must allow Administrators to review prospective artist applications, respond to user reports, and manage/ban accounts violating the Terms of Service (ToS).
- The system must allow Fans to submit appeals on their ban decisions and have an Admin other than the one who initially banned them review the appeals requests.

Non-Functional Requirements / Environment:

- The system must be a web-based application utilizing a 3-tier architecture (frontend web server, backend application layer, and database) to handle persistence and advanced business logic.
- The system must be deployed to a cloud hosting service such as Google Cloud Platform (GCP), Amazon Web Services (AWS), or Microsoft Azure.
- The development lifecycle must incorporate a CI/CD pipeline with mandatory pull requests.
- Any optional AI functionalities (e.g., content moderation, recommendation engines, or AI-generated art detectors) must be designed to respect user privacy, include human oversight, and feature fallback behavior.

4. User Story

- **[US-01]** As a **Fan**, I want to **search for artists using a proximity-based radius** so that **I can discover and attend local shows**. *(Maps to UC: Search for Artist)*
- **[US-02]** As a **Fan**, I want to **receive artist recommendations** so that **I can easily find new creators similar to the ones I already enjoy**. *(Maps to UC: Receive Recommendation)*
- **[US-03]** As an **Artist**, I want to **create posts with multimedia content (music, images, videos)** so that **I can share my art and stories with my fan community**. *(Maps to UC: Make a Post)*
- **[US-04]** As a **Fan**, I want to **comment on an artist's uploaded artwork or posts** so that **I can show my support and engage with the community**. *(Maps to UC: Comment on Artwork)*
- **[US-05]** As an **Artist**, I want to **monitor and manage the comments on my posts** so that **I can hide inappropriate content and report abusive users to admins**. *(Maps to UC: Monitor Comments)*
- **[US-06]** As a **User (Fan or Artist)**, I want to **send direct messages to other users** so that **I can privately communicate, collaborate, or socialize**. *(Maps to UC: Message Another User)*
- **[US-07]** As an **Artist**, I want to **create and promote upcoming events** so that **my fans know when and where to see my shows**. *(Maps to UC: Create Event)*
- **[US-08]** As an **Artist**, I want to **edit and update my existing event information** so that **my fans have the most accurate and current details if plans change**. *(Maps to UC: Update Event)*
- **[US-09]** As an **Administrator**, I want to **approve prospective artist verification applications** so that **I can ensure only legitimate, non-fake artists are granted the Artist role**. *(Maps to UC: Approve Artist Verification Application)*
- **[US-10]** As an **Administrator**, I want to **review conduct reports submitted by users** so that **I can make fair moderation decisions regarding inappropriate content**. *(Maps to UC: Review Report)*
- **[US-11]** As an **Administrator**, I want to **suspend or ban user accounts that violate the terms of service** so that **the platform remains a safe environment for artists and fans**. *(Maps to UC: Suspending Artist Account)*

- **[US-12]** As an **Administrator**, I want to **review and process ban appeals from fans** so that **I can reverse unfair or incorrect bans made by the system or other admins**. *(Maps to UC: Unban Fan)*

5. Key Technical Challenges

- **AI Feature Integration:** Integrating machine learning for moderation and recommendations poses performance and privacy challenges.
  Approach: Decouple AI processing from core traffic using scalable REST APIs and implement heuristic fallbacks to ensure uptime if models fail.
- **3-Tier Cloud Deployment:** Managing a frontend, backend, database, and strict CI/CD pipeline requires significant DevOps overhead.
  Approach: Establish automated testing and mandatory pull requests early, utilizing managed cloud services (GCP, AWS, or Azure) to minimize infrastructure management.

Key Risks

- **Scope Creep:** The extensive feature set (hybrid media, messaging, AI) risks exceeding the semester timeframe.
  Mitigation: Strictly prioritize core workflows (authentication, media, messaging) over optional AI features using agile sprints.
- **Content Moderation:** User-generated media and messaging introduce risks of Terms of Service violations.
  Mitigation: Implement robust user-reporting systems, Admin review workflows, and automated moderation filters.

Competition

- **SoundCloud:** *What it does well:* Excellent audio hosting, discovery, and timestamped comments.
  Where it falls short: Lacks visual art hosting, video interactions, and proximity-based local discovery.
- **Instagram:** *What it does well:* Massive reach for visual art, short-form videos, and messaging.
  Where it falls short: Not tailored to independent artists, lacks high-quality audio hosting, and prioritizes viral content over local community connections.

6. What's New?

Existing platforms generally focus on a single medium (e.g., SoundCloud) or cater to broad audiences (e.g., Instagram). Our project introduces a **hybrid platform tailored explicitly for small, independent, multi-disciplinary artists**.

Key Differentiators:

- **Multi-Disciplinary Hub:** Consolidates high-quality audio, visual art, and video hosting into a single profile.
- **Proximity-Based Discovery:** Replaces viral-chasing algorithms with a location-based search to foster real-world, local community connections.
- **Authenticity & Safety Focus:** Proposes specialized AI tools—like AI-generated art detectors and privacy-respecting moderation—to maintain a secure, human-centric ecosystem dedicated to independent art.

7. Use Cases

Actors Overview

- **Fan** – discovers and follows artists, messages them, and books events.
- **Artist** – manages a public profile, proposes events, and communicates with fans.
- **Admin** – moderates content and accounts by reviewing user-filed reports.

7.1 Alejandro Garcia Jr – Use Cases

7.1.1 UC: Search for Artist – Fan

Primary Actor
Fan

Scope ArtistConnect chat application

Level User goal

Stakeholders & Interests
- **Fan:** wants to quickly find artists matching their interests, including artists close by.
- **Artist:** wants to be discoverable by relevant fans in their area.

Preconditions
- The Fan is authenticated with a Fan account in good standing. Artists and Admins cannot perform this use case.

Postconditions (Success Guarantee)
- A list of artists matching every criterion the Fan entered was displayed.
- If the Fan chose a distance, only artists within that distance were listed, nearest first, each with its distance.
- No system data was created or changed.

Main Success Scenario
1. The Fan selects the search feature.
2. The system prompts the Fan for search criteria: name, genre/style, location, and an optional distance (e.g., "within 25 km of me").
3. The Fan enters one or more criteria and submits the search.
4. The system finds registered artists matching all of the given criteria.
5. The system displays the matching artists, each showing name, genre, location, profile thumbnail, and distance when a distance was chosen.

Extensions *a. The user is not signed in with an active Fan account: the system denies the search.

3a. The Fan submits the search with no criteria entered: the system prompts the Fan to enter at least one criterion.

3b. The Fan chooses a distance and shares their current location: the system searches around the shared location instead of the Fan's saved home location.

3c. The Fan chooses a distance but has neither shared a location nor saved a home location: the system asks the Fan to share their location or save a home location.

4a. No artists match the given criteria: the system displays a "no results found" message and suggests broadening the search (e.g., a larger distance).

Special Requirements
- Search results should be returned in under 2 seconds for typical query loads.
- Result lists longer than 20 artists are paginated.
- A Fan's location is used only to run the search and is never shown to other users.

Frequency of Occurrence
High – expected to be one of the most common Fan actions.

Notes Viewing an artist's profile, following, messaging, or booking an artist from the results are separate use cases.

7.1.2 UC: Review Report – Admin

Primary Actor
Admin

Scope ArtistConnect chat application

Level User goal

Stakeholders & Interests
- **Admin:** needs enough context to make a fair, auditable moderation decision.
- **Reporting user (Fan or Artist):** wants inappropriate content or behavior addressed.
- **Reported user (Fan or Artist):** has an interest in fair treatment and being informed of any action taken.

Preconditions
- The Admin is authenticated with administrative privileges.
- At least one report exists with status "Pending," filed by a Fan or an Artist against a message, profile, or event.

Postconditions (Success Guarantee)
- The report's status was updated to "Resolved," with the decision and acting Admin recorded.
- Any moderation action taken (dismiss, warn, delete content, suspend/ban account) was applied to the reported content or account.
- Both the reporting user and the reported user were notified of the outcome.

Main Success Scenario 1. The Admin opens the reports queue.

2. The system displays pending reports, each showing the reporter, reported content or user, reason, and timestamp.
3. The Admin selects a report to review.
4. The system displays full report details, including the reported message, profile, or event, and the reporter's stated reason.
5. The Admin evaluates the report and selects an action: dismiss, warn the user, delete the content, or suspend/ban the account.
6. The system applies the selected action.
7. The system records the resolution (Admin identity, action taken, timestamp) and sets the report status to "Resolved."
8. The system notifies the reporting user that their report was reviewed, and notifies the reported user of any action taken against them.

Extensions *a. The user is not signed in with an active Admin account: the system denies access to the reports queue.

3a. The selected report was already resolved by another Admin: the system informs the Admin and returns to the queue.

5a. The Admin dismisses the report as unfounded: the system records the dismissal and notifies only the reporting user.

5b. The Admin suspends or bans the reported user's account: the account can no longer perform any Fan or Artist actions until it is restored.

5c. The Admin chooses to delete content on a report against a user profile: the system explains that only warn, suspend, or ban apply to user reports.

6a. The reported content was already removed (e.g., deleted by its owner): the system notes this and proceeds directly to logging the resolution.

Special Requirements
- The reports queue must support filtering by report type (message, profile, or event).
- All resolutions must be permanently logged for audit purposes.

Frequency of Occurrence
Low to moderate – driven by user-filed reports rather than a routine action.

7.1.3 UC: Create Event – Artist

Primary Actor
Artist

Scope ArtistConnect chat application

Level User goal

Stakeholders & Interests
- **Artist:** wants to promote and sell tickets to an upcoming event.
- **Admin:** wants submitted events to be complete and valid before reviewing their pricing.
- **Fan:** wants accurate, trustworthy event listings.

Preconditions
- The Artist is authenticated with an Artist account in good standing (not suspended or banned) and has an artist profile. Fans and Admins cannot perform this use case.

Postconditions (Success Guarantee)
- A new event was created, linked to the Artist, with status "Pending Approval" (or "Draft" if the Artist saved it as a draft).
- The event appears in the Artist's list of events.

Main Success Scenario
1. The Artist selects "Create Event."
2. The system displays the event creation form.
3. The Artist enters the event details: title, date/time, location (physical address or virtual link), description, capacity, and one or more ticket tiers with prices.
4. The Artist submits the event.
5. The system validates the submitted data.
6. The system creates the event with status "Pending Approval."
7. The system confirms to the Artist that the event was submitted for review.

Extensions
*a. The user is not signed in with an active Artist account: the system denies the request.

4a. The Artist saves the event as a draft instead of submitting: the system stores it with status "Draft" and confirms the draft was saved.

5a. Required fields are missing, the date is not in the future, the capacity is not positive, or a price is negative: the system flags the offending fields and asks the Artist to correct them, returning to step 3.

5b. The event is marked virtual but the location is not a valid web link: the system asks for a valid link, returning to step 3.

Special Requirements
- Ticket pricing must support multiple tiers (e.g., general admission, VIP).

Frequency of Occurrence
Moderate – expected whenever an Artist plans a new event.

Notes
An Admin reviewing and publishing the event is a separate use case (Review Event Pricing).

Repeat the section above for each additional team member, with their own 3 use cases in the same fully-dressed format.

7.2 Garrett DeYong – Use Cases

7.2.1 UC: Make a Post – Artist

Use case: Make a post
Actor: Artist
Preconditions:

1. The artist is logged in.
2. The artist is allowed to post.

Postconditions;

1. A new post belonging to the user is saved in the database.
2. This post can be viewed and commented on according to provided settings.

Main flow:

1. The artist selects the option to create a post.
2. The system prompts the artist to provide post content (text, files) and post permissions (whether it is public, whether it can be commented on).
3. The artist provides this information.
4. The system stores the post.
5. The system adds the post to the artist's account.

Extensions:

4a. No post content provided

4a1. The system determines no post content was provided.
4a2. The system prompts the sender to provide post content.

Special requirements:

- For public posts, the system should update the artist's profile content quickly (within 5 seconds, ideally).

7.2.2 UC: Message Another User (Beginning a New Conversation)

Use case: Message another user (beginning a new conversation)
Actor: User (artist or fan)
Preconditions:

1. The sender is logged in.
2. The sender is allowed to send messages.
3. The sender has accessed the recipient's page.
4. The recipient is accepting messages (both in general and from the sender specifically).
5. The sender and recipient do not currently have an ongoing conversation.

Postconditions:

1. A new conversation thread is established.

Main flow:

1. The sender selects the option on the recipients profile to begin a message.
2. The system prompts the sender to type the message.
3. The sender inputs the message.
4. The system checks and validates the message.
5. The system creates a new conversation thread with the provided message.
6. The system enables access to this thread to the sender and the recipient.

Extensions:

4a. Message empty

4a1. The system determines no message was provided.
4a2. The system prompts the sender to type the message.

4b. Message not appropriate/allowable

4b1. The system determines the message is not appropriate in some way.
4b2. The system warns the sender of this and prompts for message modifications.

Special requirements:

- The recipient should be notified that they have received a new message.

7.2.3 UC: Monitor Comments – Artist

Use case: Monitor comments
Actor: Artist
Preconditions:

1. The artist is logged in.
2. The artist has accessed the comment section of one of their posts.

Postconditions:

1. A given comment has been hidden from users.
2. Admins have been informed, if the artist so chooses.
3. The comment's author has been blocked, if the artist so chooses.

Main flow:

1. The artist selects the manage option on a comment.
2. The system prompts the artist to select the desired action (delete/report/block user).
3. The artist provides this information.
4. The system hides the comment from users.

Extensions:

4a. Report option selected

4a1. The system files a report after hiding the comment, to be viewed by admins.

4b. Block option selected

4b1. The system makes a request to prevent the user from interacting with the artist's posts in the future.

Special requirements:

- The system should not delete the comment from the database; it should be stored, but not visible to non-admin users.
- The comment should be hidden quickly (within 2 seconds, ideally).
- The user should be blocked (if applicable) quickly (within 2 seconds, ideally).
- The report (if applicable) should be sent within a few minutes.

7.3 Nicholas Nolen – Use Cases

7.3.1 UC: Update Event

Primary Actor
Artist

Scope Artist Home Page

Level User goal

Stakeholders & Interests
- **Fan:** As a fan I want to be able to view artist's most up to date event information.
- **Artist:** As an artist I want to provide up to date information about my events.

Preconditions
- An artist has logged into the system.
- An artist has already created an event.

Postconditions (Success Guarantee)
- An event has been updated on the artist's profile.

Main Success Scenario
1. The artist requests to modify an existing event.
2. The system prompts the artist to write over the event information (e.g. event type, description, date, time, relevant images/flyers).
3. The artist updates their event's information.
4. The system validates the input.
5. The system prompts the artist to review their updated event details.
6. The artist acknowledges they reviewed their details and submits the event.
7. The event shows up on the artist's profile to everyone who views it.

5a. Extensions Invalid input

- The system detects invalid input.
- The system displays an error message and requests the artist correct the issue.

6a. Artist fails to acknowledge

- The system displays an error message stating the artist must acknowledge they have reviewed the details of their submission before submitting.
- The artist acknowledges their review.
- The artist submits their event.

Frequency of Occurrence

Mdderate – It is expected for artists to make mistakes uploading event information.

7.3.2 UC: Unban Fan

Primary Actor

Admin

Scope Administrative service

Level System goal

Stakeholders & Interests

- **Admin:** As an admin I want to do my job and process appeals
- **Fan:** As a fan I want to be appealed to access the site.

Preconditions

- An Admin has logged into the system.
- A ban appeal has been made by a fan.

Postconditions (Success Guarantee)

- A different admin than the banning admin has reviewed the appeal.
- The appeal request has been processed and the fan is unbanned.

Main Success Scenario

1. The admin selects the option to review open appeals from bans performed by admin other than themselves.
2. The admin selects an appeal.
3. The admin reviews fan submitted reasoning for the appeal along with related report material along with an automated AI recommendation based on the ToS.
4. The admin selects an option to proceed with unbanning the user.
5. The system removes the user's ban.
6. The system grants the user the ability to log into the website.

Extensions 1a. No appeals

- The system shows no open appeals.
- The admin closes the reports tab.

5a. Appeal is invalid

- The admin selects an option to mark the appeal as invalid.
- The system logs the appeal as invalid and removes it from the open appeals list.

Frequency of Occurrence

High – There will likely always be fans trying to appeal bans.

7.3.3 UC: Recieve Reccomendation

Primary Actor
Fan

Scope Fan Home Page

Level User goal

Stakeholders & Interests
- **Fan:** As a fan I want to be able to find artists similar to those I like.
- **Artist:** As an artist I want to get my art out there to fans.

Preconditions
- A fan has logged into the system.
- A fan has previously interacted with artists on the platform.

Postconditions (Success Guarantee)
- A fan receives recommendations for artists they might like.

Main Success Scenario
1. A fan selects the option to display artists they might like.
2. The system retrieves the artists a fan has previously interacted with (e.g. viewed profiles, liking posts, commenting on posts, etc.).
3. The system utilizes an algorithm to determine artists similar to the fan's most interacted with artists.
4. The system displays a list of artists to the user.

Extensions 1a. New Fan
- The fan has logged into the system for the first time and does not have interactions.
- The system does not show the option to display artists they might like.

Frequency of Occurrence
High – It is likely fans will want frequent reccomendations.

7.4 Gan-Orgil Gantumur – Use Cases

7.4.1 UC: Suspending Artist Account – Admin

Use case: Suspending Artist Account
Actor: Administrator
Preconditions:

1. The Administrator is logged into the system.
2. A user account has been reported or flagged for potential terms of service (ToS) violations.

Postconditions:

1. The user's account is successfully suspended.
2. The system revokes the user's access and permissions.

Main flow:

1. The Administrator requests to view reported user accounts.
2. The system provides the pending reports and associated user activity.
3. The Administrator determines the user's actions violate the ToS.
4. The Administrator submits a request to suspend the user.
5. The system updates the account status to suspended and revokes the user's permissions.

Extensions:

3a. Report does not violate ToS

- 3a1. The Administrator determines the user's actions do not violate the ToS.
- 3a2. The Administrator dismisses the report, and the system maintains the account's active status.

7.4.2 UC: Comment on Artwork – Fan

Use case: Comment on Artwork
Actor: Fan
Preconditions:

1. The Fan is logged in with "user" role permissions.
2. An Artist has published media that allows comments.

Postconditions:

1. The Fan's comment is saved and published on the Artist's post.

Main flow:

1. The Fan requests to add a comment to an Artist's media.
2. The system prompts the Fan for the comment content.
3. The Fan provides the comment content.
4. The system validates the comment against business rules.
5. The system saves and publishes the comment to the Artist's page.

Extensions:

4a. Content flagged by system business rules

- 4a1. The system determines the content violates business rules (e.g., via basic word filters or optional AI-based content moderation, if enabled).
- 4a2. The system rejects the comment and notifies the Fan.

5a. Inappropriate comment reported post-publication

- 5a1. An Artist monitors their comments and identifies a published comment as inappropriate.
- 5a2. The Artist submits a report regarding the comment for Administrator review.

7.4.3 UC: Approve Artist Verification Application – Admin

Use case: Approve Artist Verification Application
Actor: Administrator
Preconditions:

1. The Administrator is logged in and authorized to manage accounts.
2. A user has submitted an application to upgrade to the Artist role.

Postconditions:

1. The user's application is resolved (approved or rejected).
2. If approved, the user is granted the Artist role and associated permissions.

Main flow:

1. The Administrator requests pending Artist verification applications.
2. The system provides the submitted application materials.
3. The Administrator evaluates the application to verify the applicant's identity.
4. The Administrator approves the application.
5. The system assigns the user the "Artist" role and updates their permissions.

Extensions:

3a. Application fails verification

3a1. The Administrator determines the application is illegitimate or incomplete.
3a2. The Administrator rejects the application.
3a3. The system maintains the user's current role and notifies them of the rejection.

7.5 William Waweru – Use Cases

7.5.1 UC: Follow Artist – Fan

Use case: Follow Artist
Actor: Fan
Preconditions:

1. The Fan is logged into the system.
2. The Artist has an active account that is visible to the Fan.
3. The Fan is not currently following the Artist.

Postconditions:

1. The Fan is recorded as a follower of the Artist.
2. The Artist is added to the Fan's list of followed Artists.

Main flow:

1. The Fan requests to follow an Artist.
2. The system verifies that the Artist's account is active.
3. The system verifies that the Fan is not already following the Artist.
4. The system creates a following relationship between the Fan and the Artist.
5. The system updates the Artist's follower information.
6. The system confirms that the Fan is now following the Artist.

Extensions:

2a. Artist account is unavailable.

- 2a1. The system determines that the Artist's account is suspended, banned, or otherwise unavailable.
- 2a2. The system does not create the following relationship.
- 2a3. The system informs the Fan that the Artist cannot currently be followed.

3a. Fan already follows the Artist.

- 3a1. The system detects an existing following relationship.
- 3a2. The system does not create a duplicate relationship.
- 3a3. The existing following relationship remains unchanged.

7.5.2 UC: Manage Fan Profile – Fan

Use case: Manage Fan Profile
Actor: Fan
Preconditions:

1. The Fan is logged into the system.
2. The Fan has an existing account and profile.

Postconditions:

1. Valid profile changes are stored by the system.
2. The updated information is associated with the Fan's profile.
3. Profile information is displayed according to the Fan's privacy settings.

Main flow:

1. The Fan requests to modify their profile.
2. The system provides the Fan's existing profile information.
3. The Fan provides updated profile information, such as a biography, artistic interests, supported Artists, or profile image.

4. The Fan specifies applicable privacy preferences.
5. The Fan submits the updated profile information.
6. The system validates the submitted information according to profile and content rules.
7. The system stores the valid changes.
8. The system applies the Fan's privacy preferences to the updated information.
9. The system confirms that the profile has been updated.

Extensions:

6a. Invalid profile information.

6a1. The system identifies invalid profile information.
6a2. The system rejects the affected changes and informs the Fan which information must be corrected.
6a3. The Fan provides corrected information.
6a4. The process resumes at Step 5.

6b. Profile content violates platform rules.

6b1. The system determines that the submitted content violates platform content rules.
6b2. The system rejects the prohibited content.
6b3. The system informs the Fan that the profile cannot be updated with that content.

7a. Profile update cannot be stored.

7a1. The system preserves the previously stored profile information.
7a2. The system informs the Fan that the update was unsuccessful.

7.5.3 UC: Report Inappropriate Content – Fan

Use case: Report Inappropriate Content
Actor: Fan
Preconditions:

1. The Fan is logged into the system.
2. The content or account being reported exists and is accessible to the Fan.

Postconditions:

1. A new report is stored with a pending status.
2. The report contains the reporting Fan, reported content or account, reason, and relevant information.
3. The report is available for Administrator review.

Main flow:

1. The Fan identifies content or user behavior that they believe violates the platform's rules.
2. The Fan requests to report the content or account.
3. The system identifies the content or account associated with the report.
4. The system requests a report reason and any additional relevant information.
5. The Fan provides a reason and submits the report.
6. The system validates that the report contains the required information.
7. The system creates the report and records the relevant information.
8. The system assigns the report a pending status and makes it available for Administrator review.
9. The system confirms to the Fan that the report has been submitted.

Extensions:

3a. Reported content is no longer available.

- 3a1. The system determines that the reported content is no longer publicly available.
- 3a2. If sufficient information about the original content remains available, the system allows the report to continue.
- 3a3. Otherwise, the system informs the Fan that the report cannot be completed.

6a. Report does not contain required information.

- 6a1. The system determines that required report information is missing.
- 6a2. The system does not submit the report.
- 6a3. The system requests the missing information from the Fan.

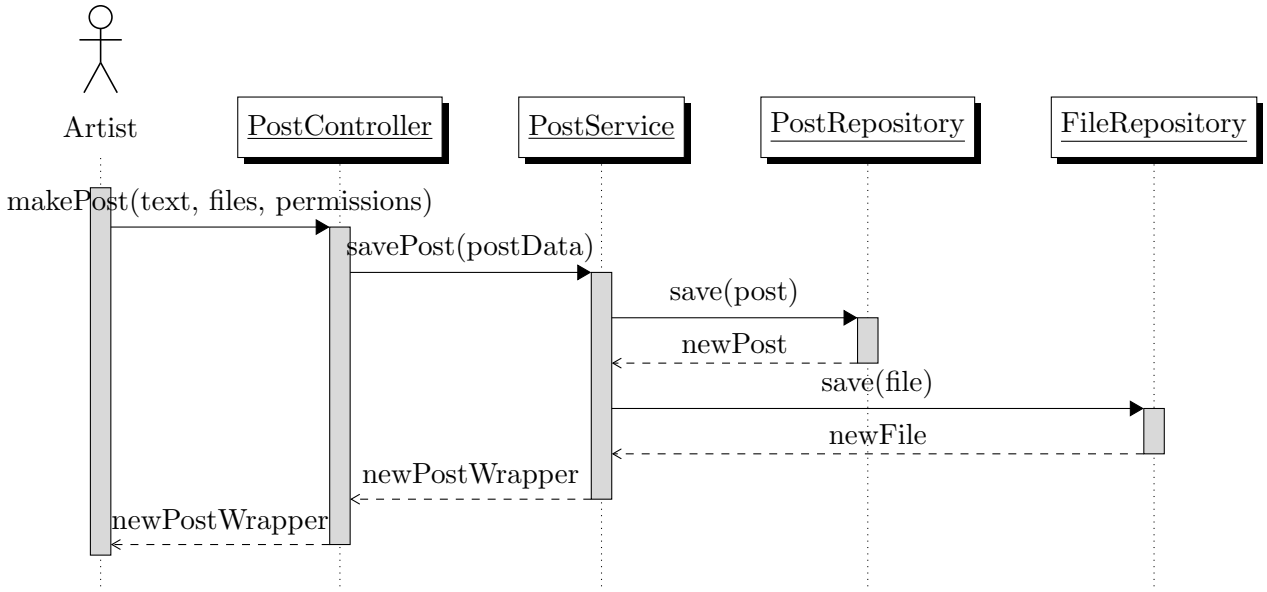
7a. Duplicate report exists.

- 7a1. The system identifies an existing pending report from the same Fan for the same content or account.
- 7a2. The system prevents creation of a duplicate report.
- 7a3. The system informs the Fan that they have already reported the content or account.

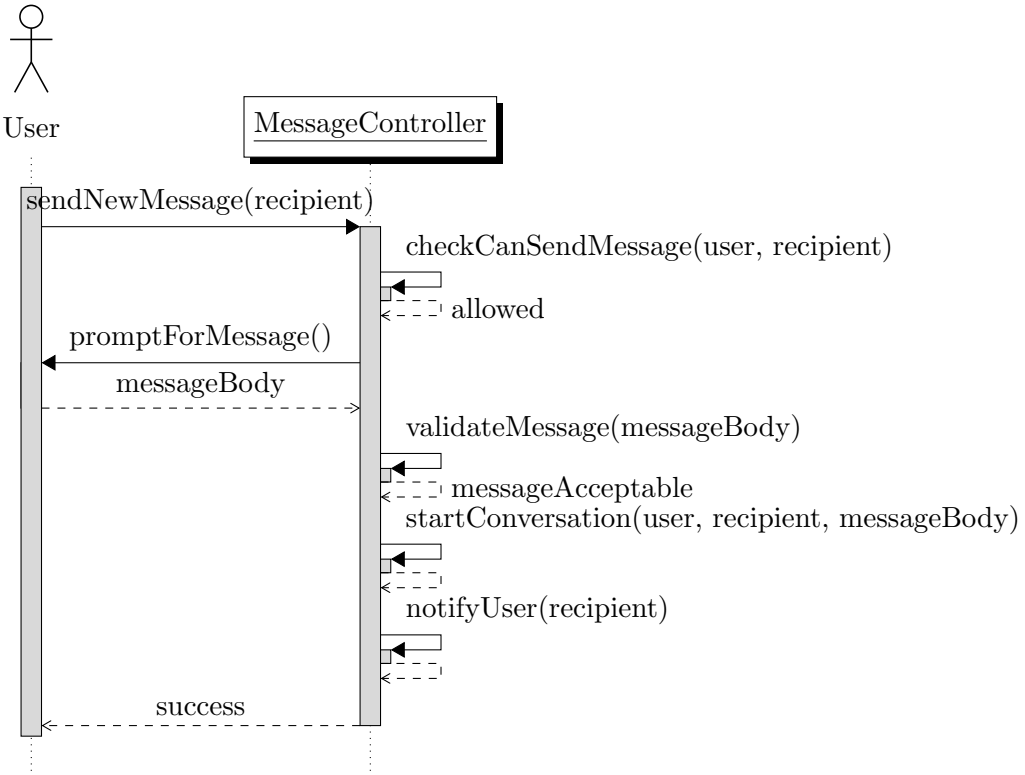
8. Sequence Diagrams

8.1 Garrett DeYong – Sequence Diagrams

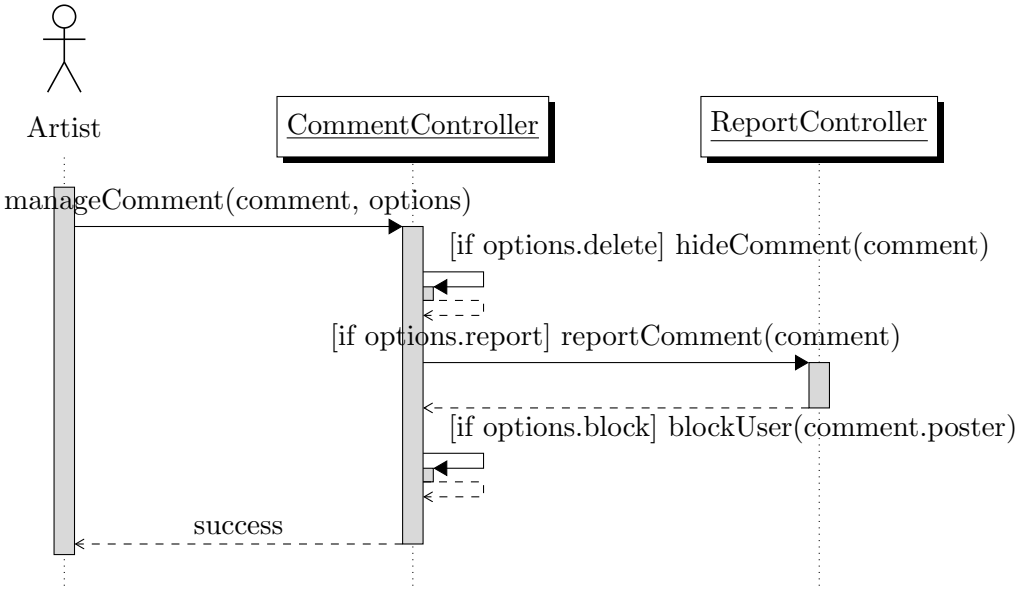
Artist - Make a post



Artist/Fan - Message another user

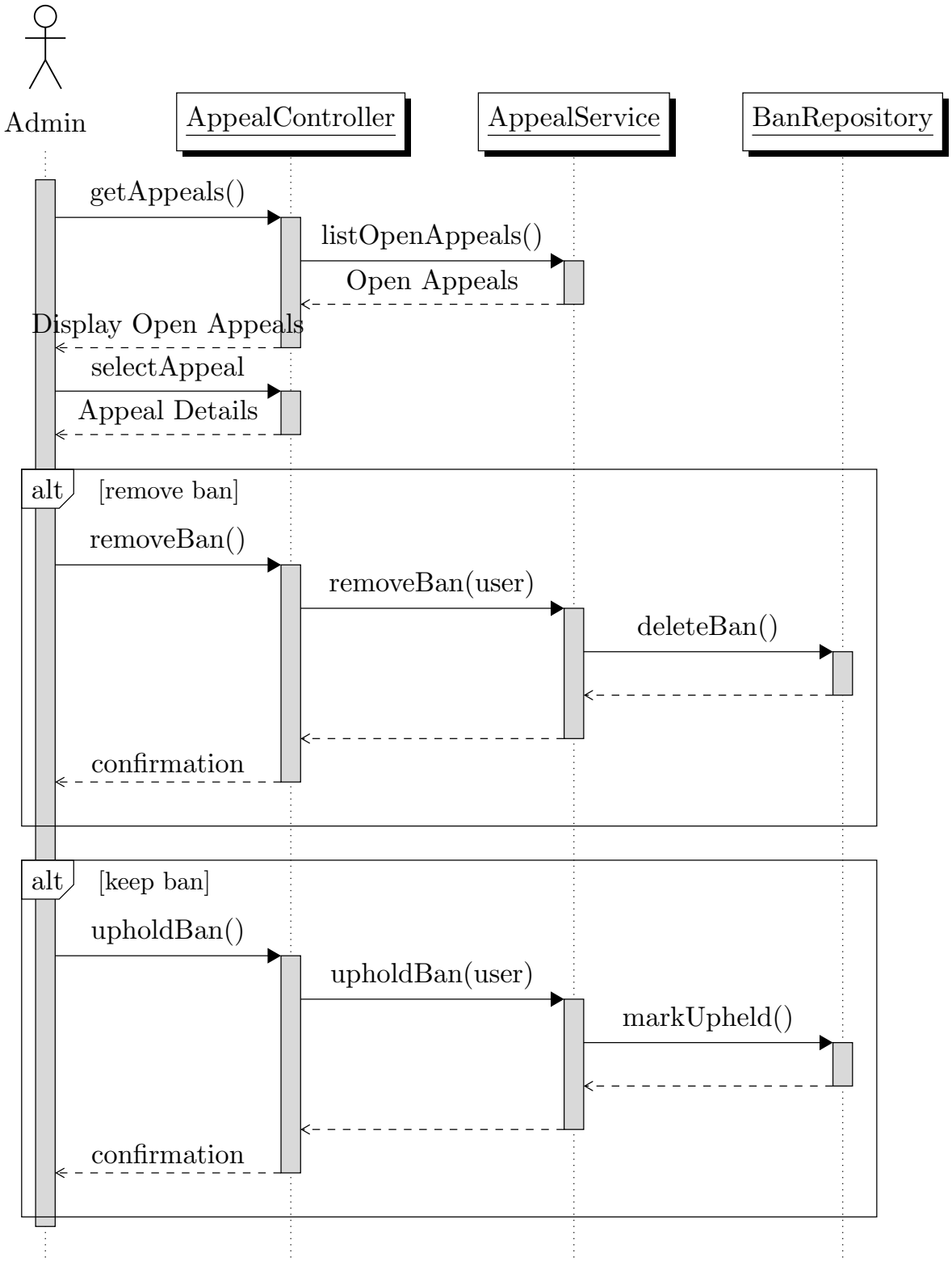


Artist - Monitor comments

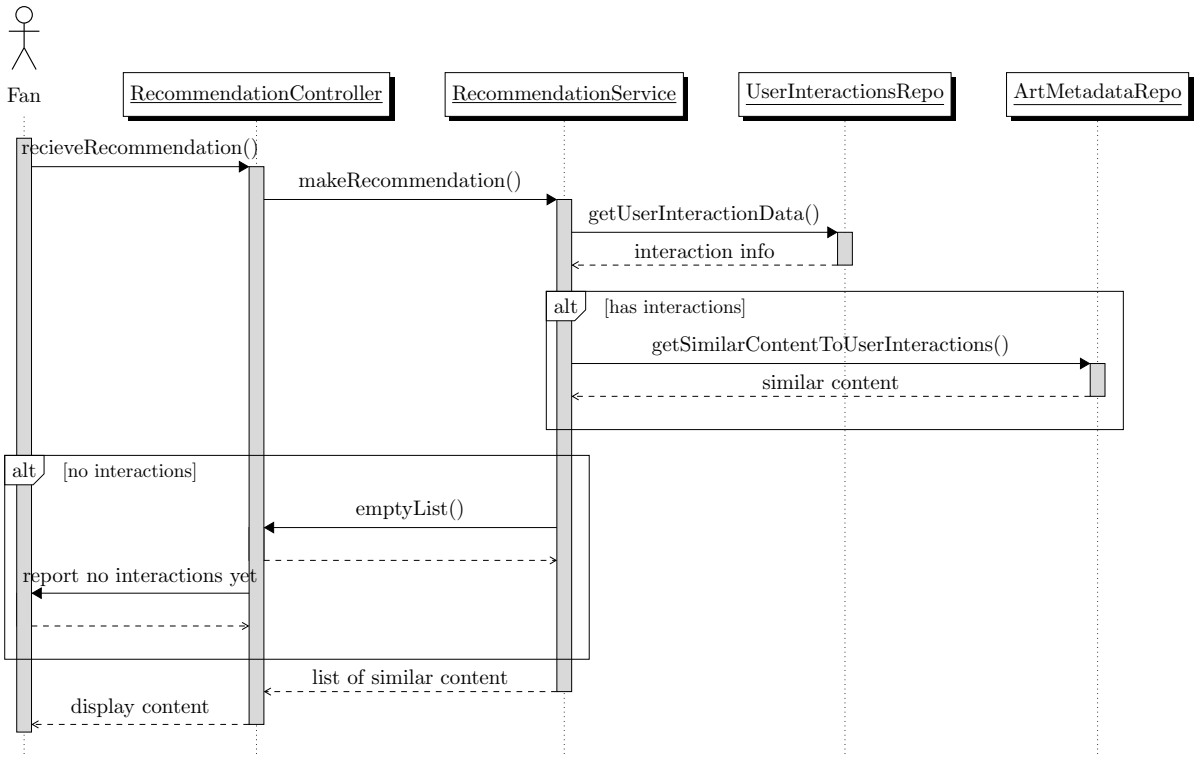


8.2 Nicholas Nolen – Sequence Diagrams

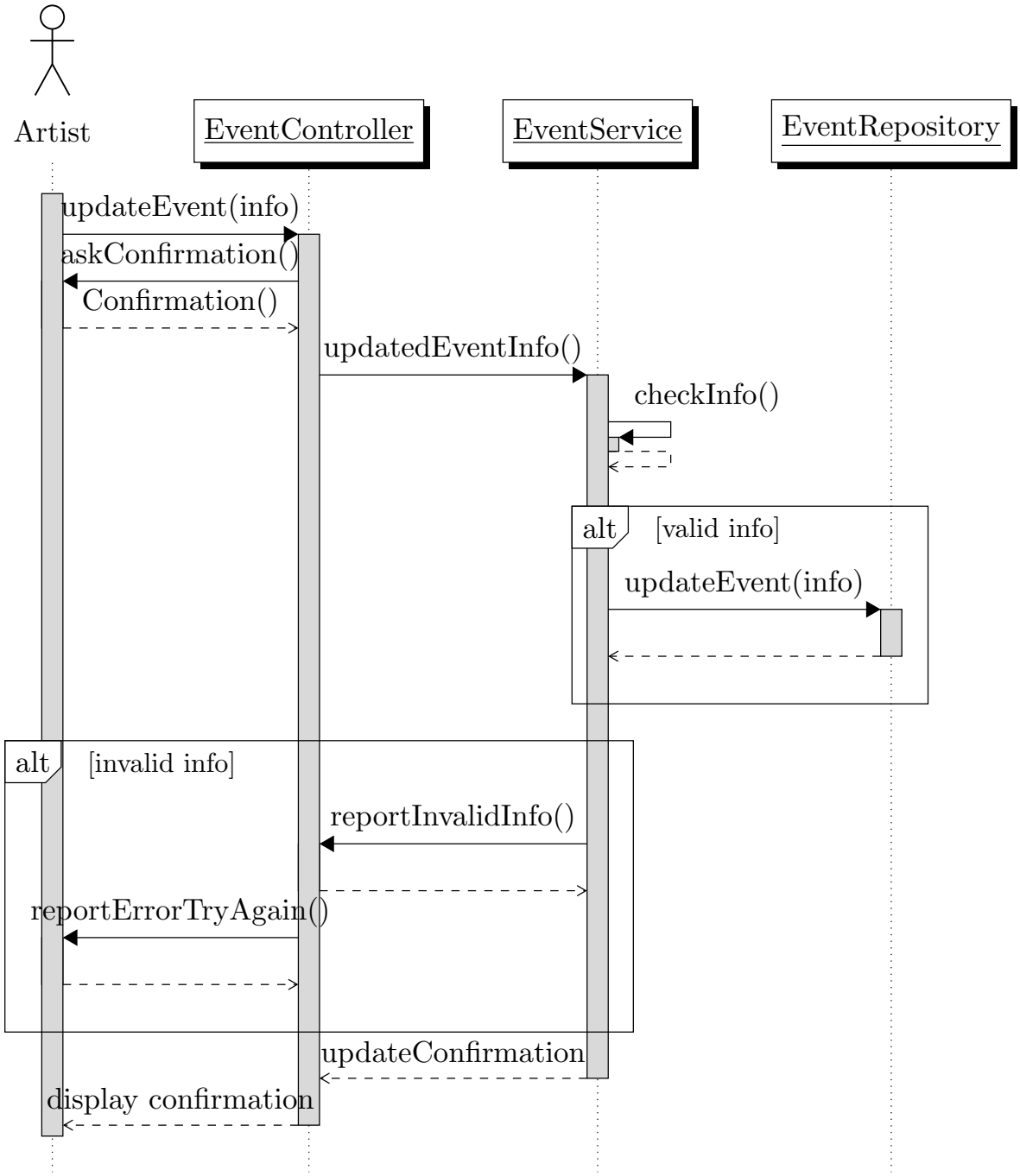
Admin - Unban Fan (Appeals)



Fan - Receive Recommendation

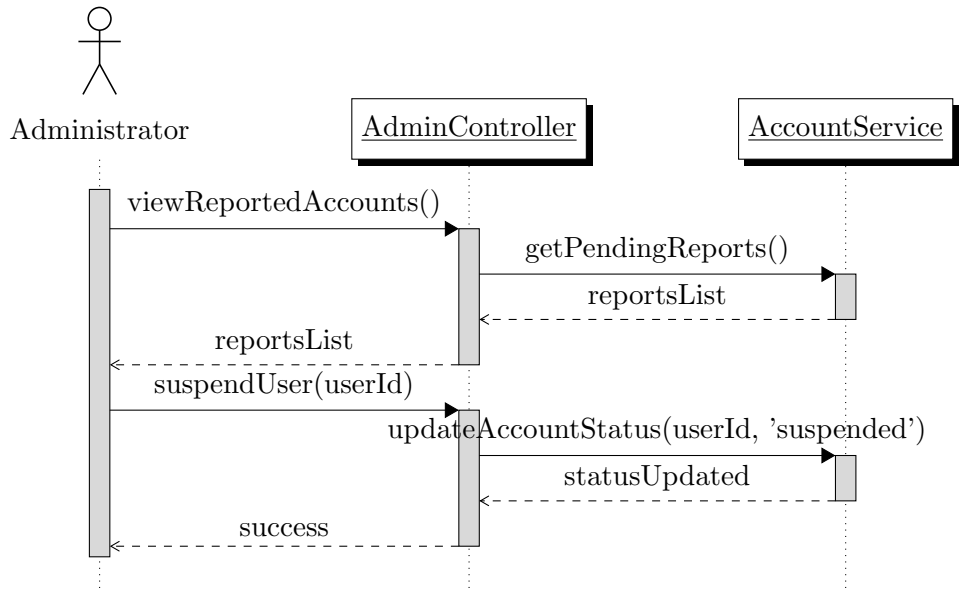


Artist - Update Event

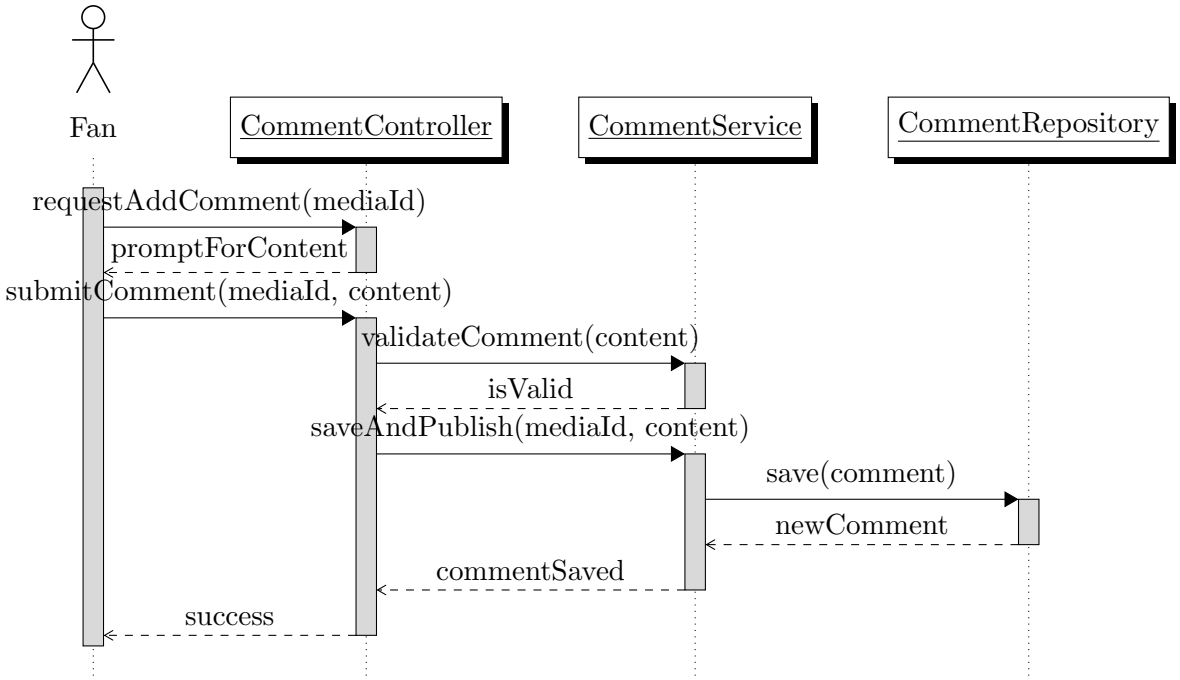


8.3 Gan-Orgil Gantumur – Sequence Diagrams

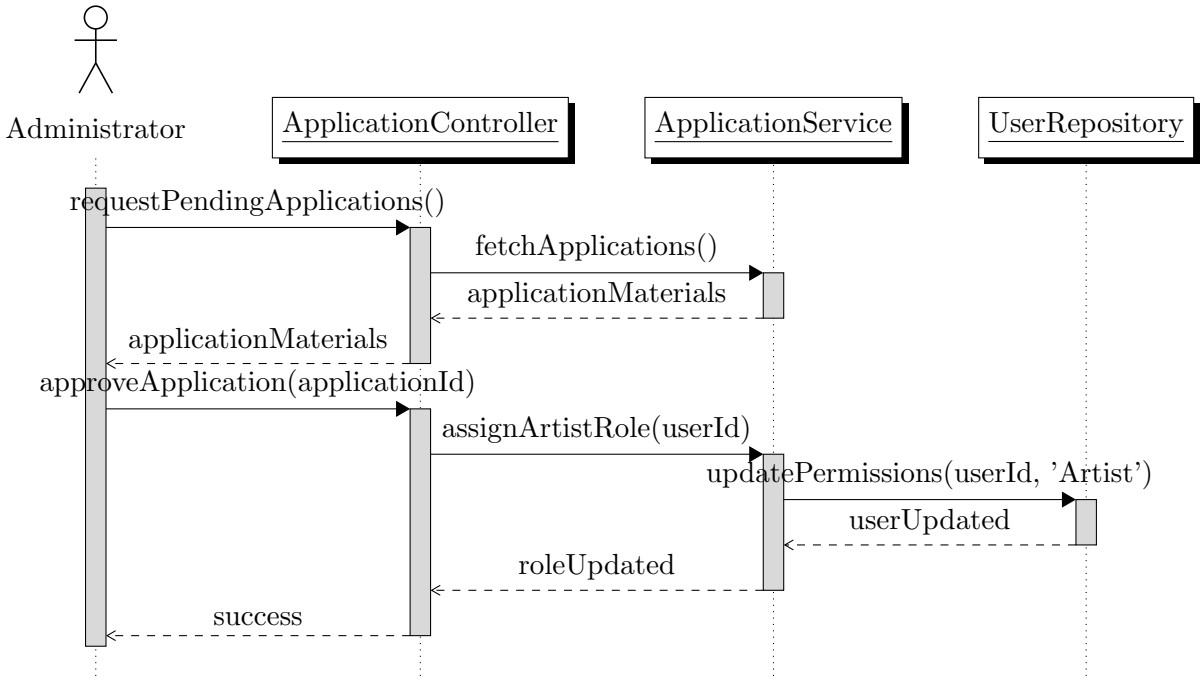
Administrator - Suspending Artist Account



Fan - Comment on Artwork

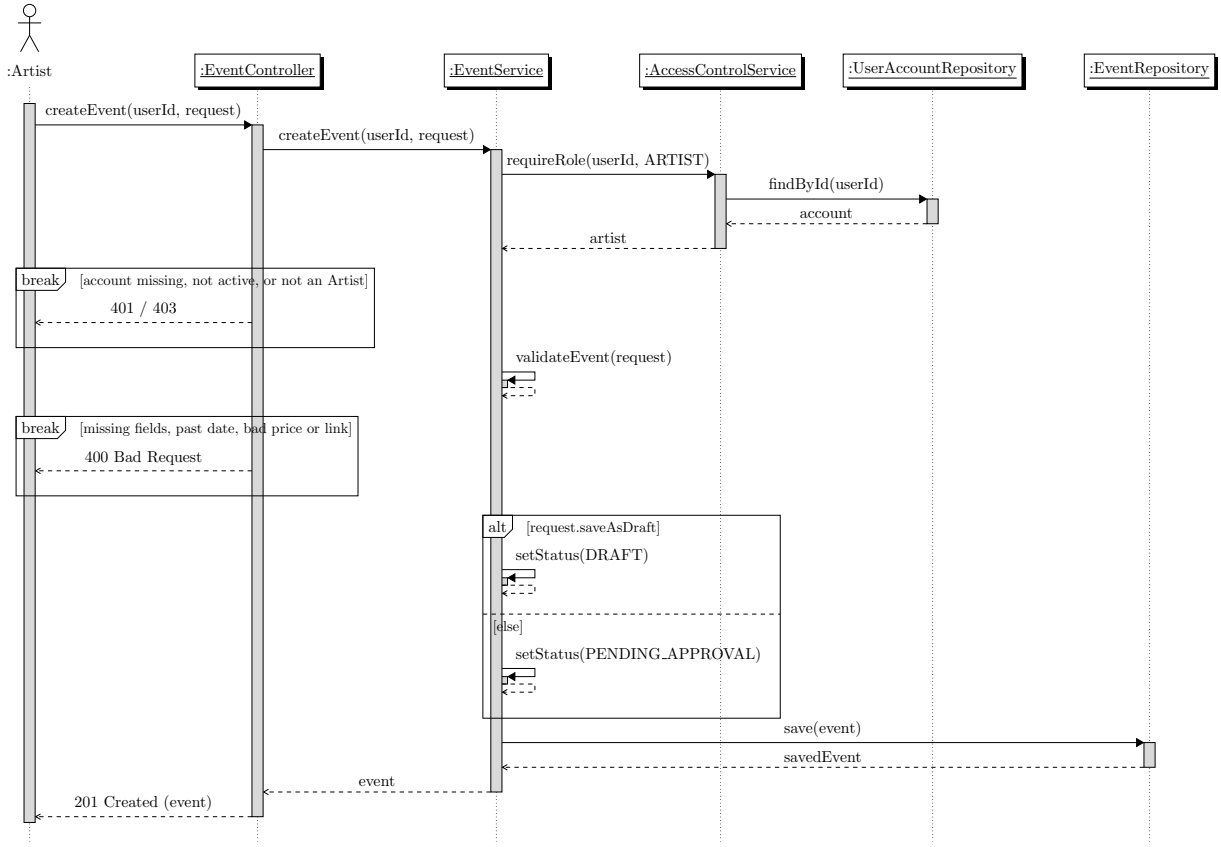


Administrator - Approve Artist Verification Application

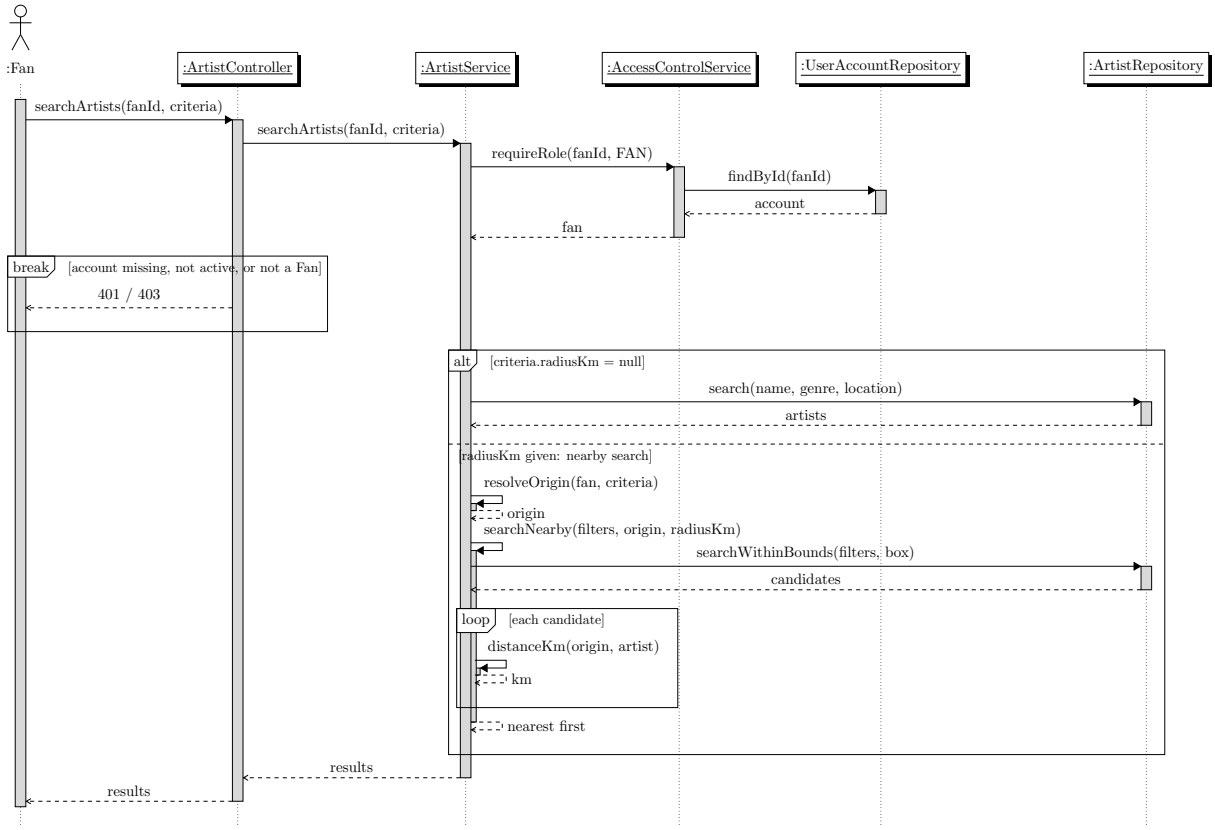


8.4 Alex Garcia – Sequence Diagrams

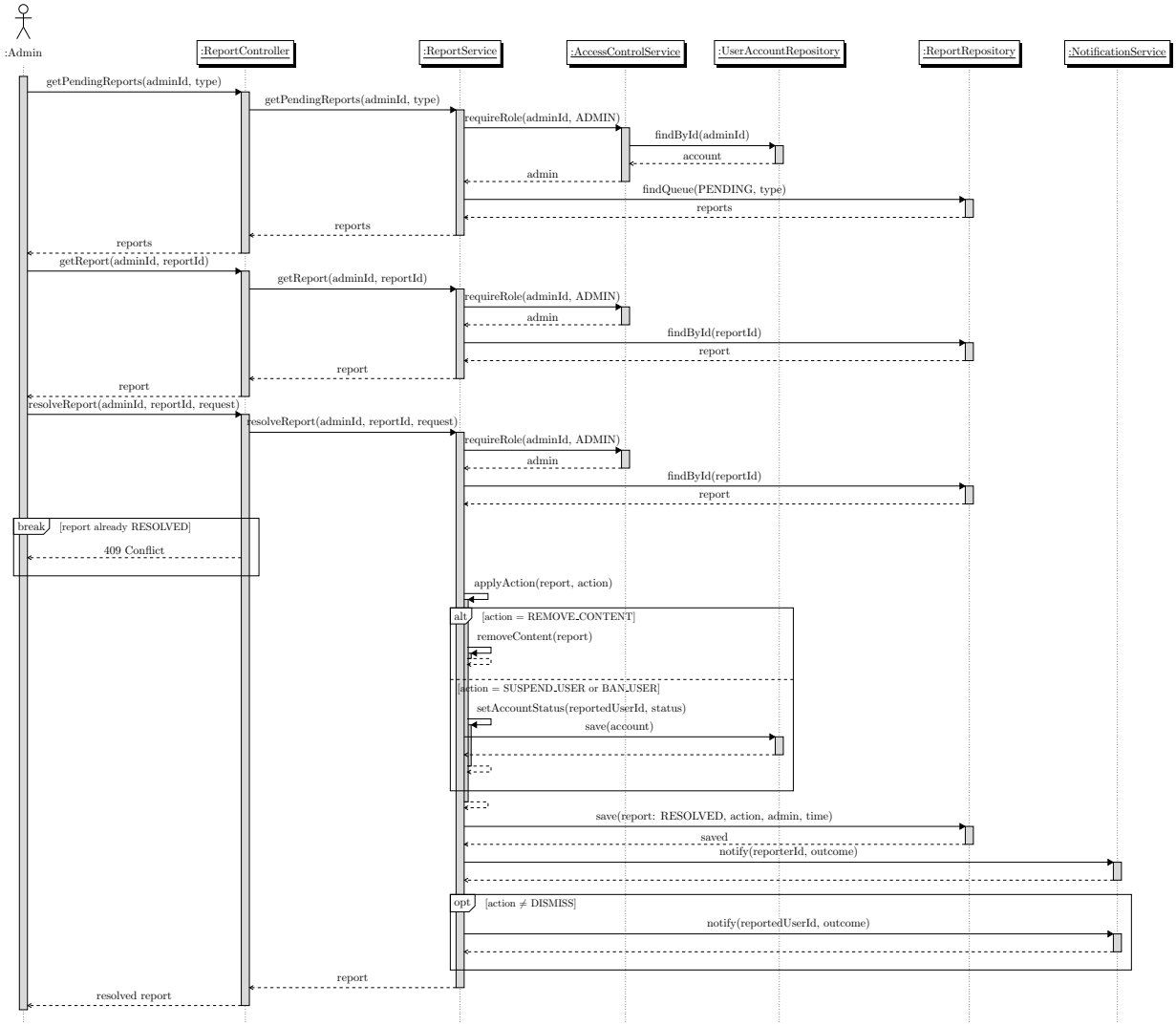
Artist – Create Event



Fan – Search for Artist

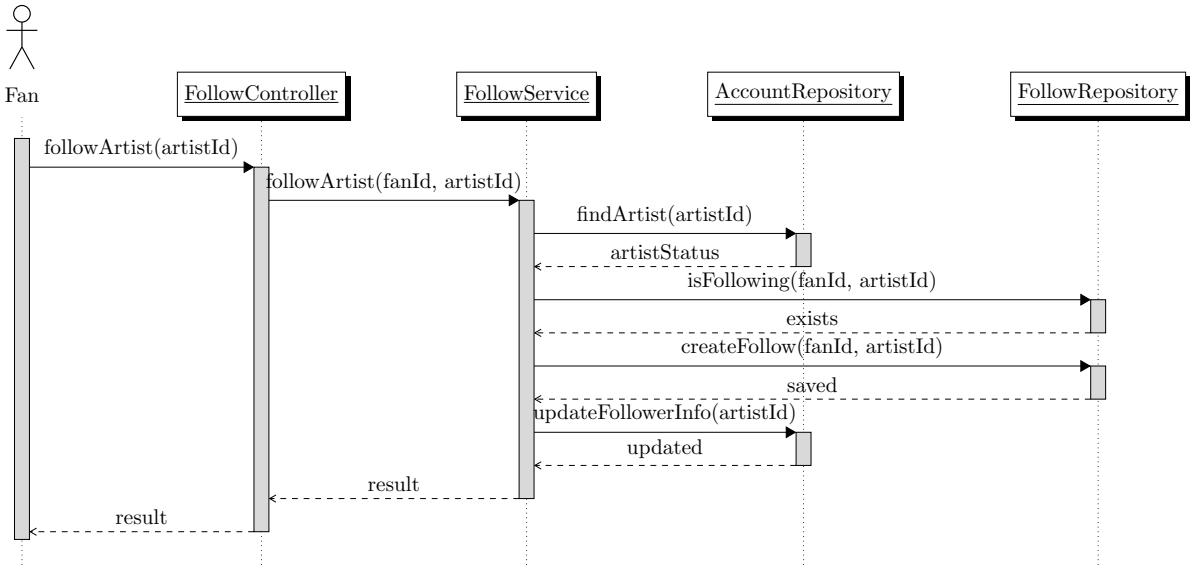


Admin – Review Report

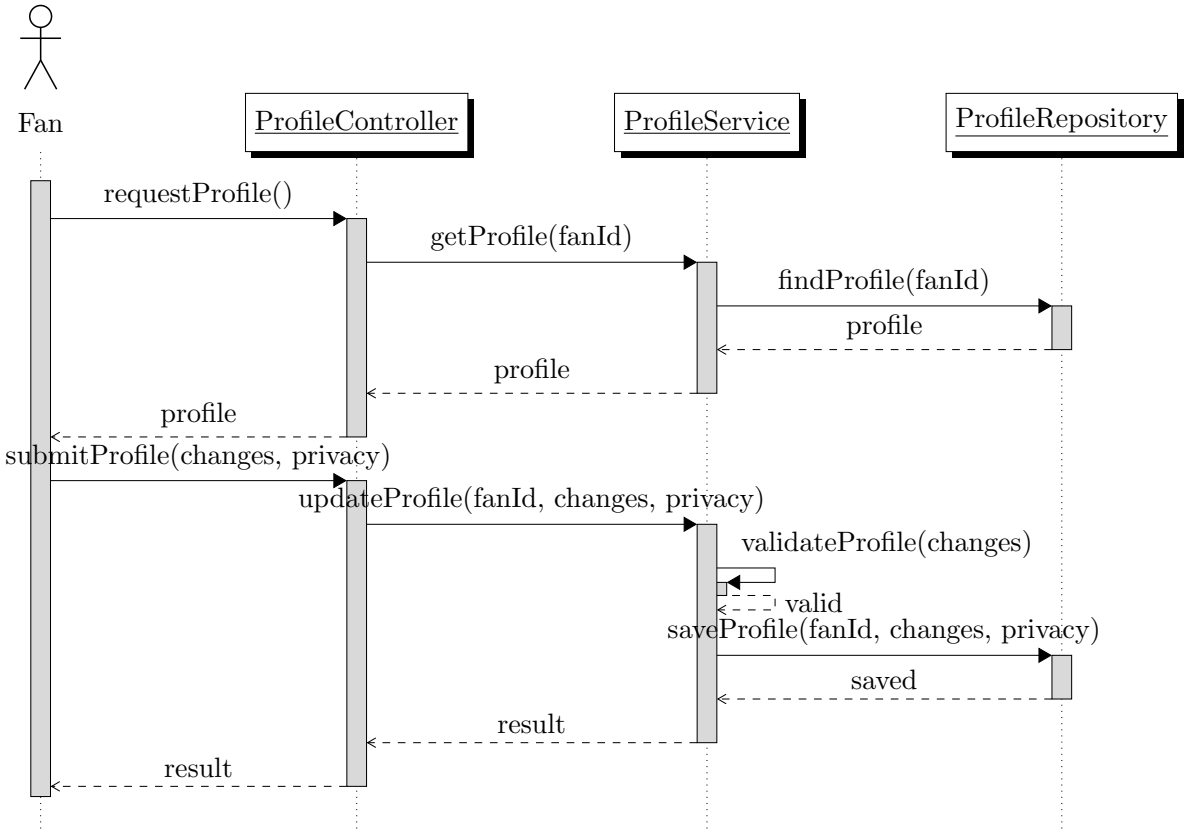


8.5 William Waweru – Sequence Diagrams

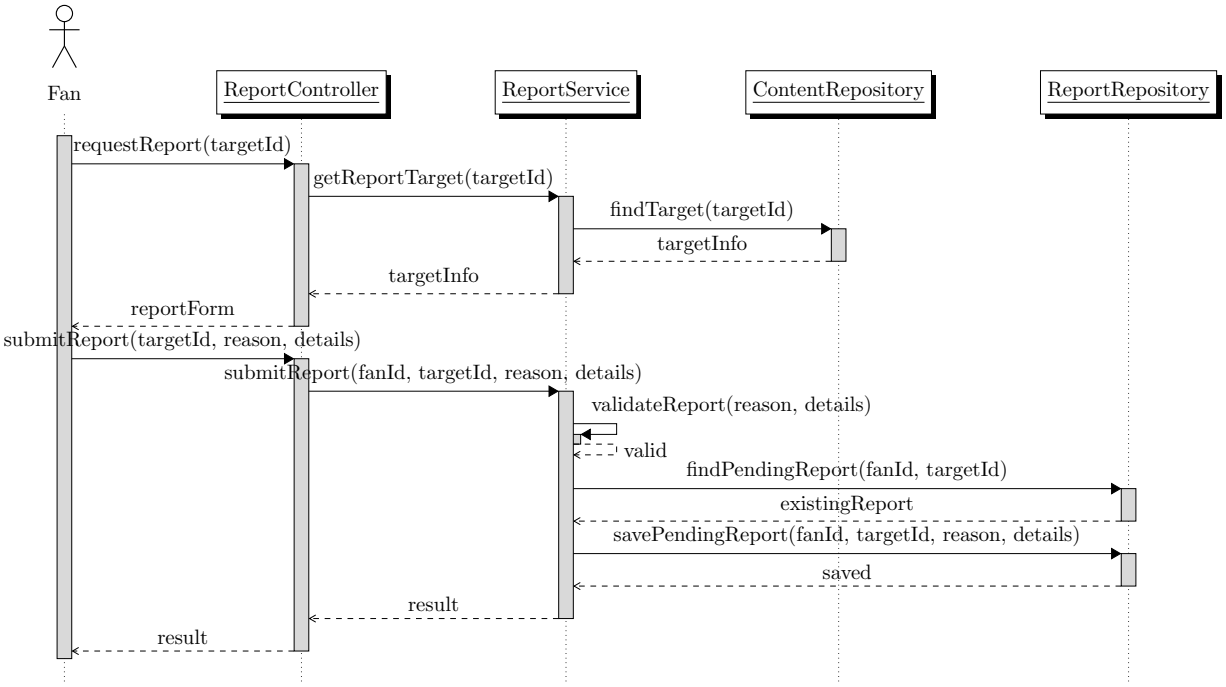
8.5.1 Follow Artist – Fan



8.5.2 Manage Fan Profile – Fan



8.5.3 Report Inappropriate Content – Fan



9. Use Case Diagram

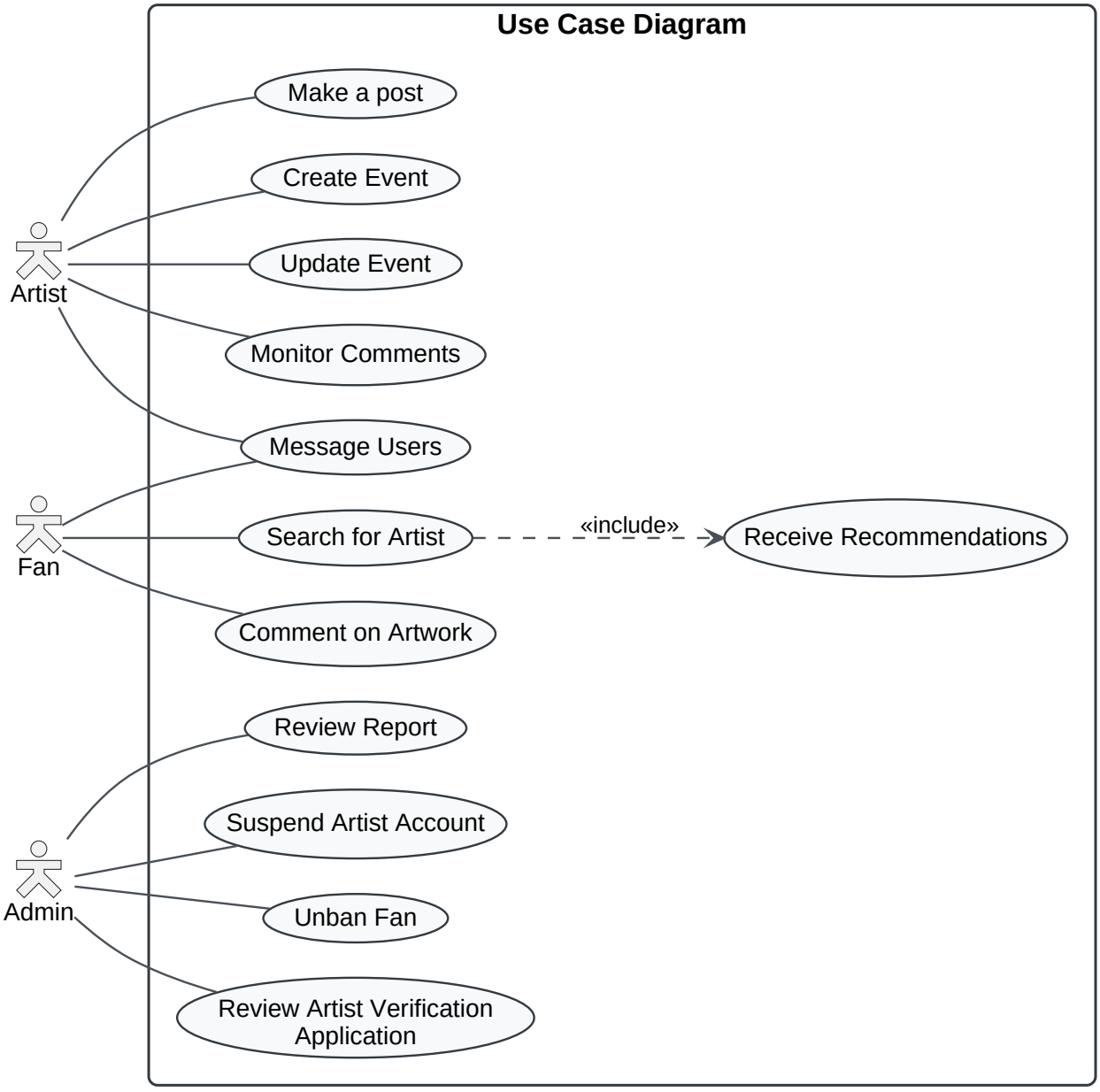
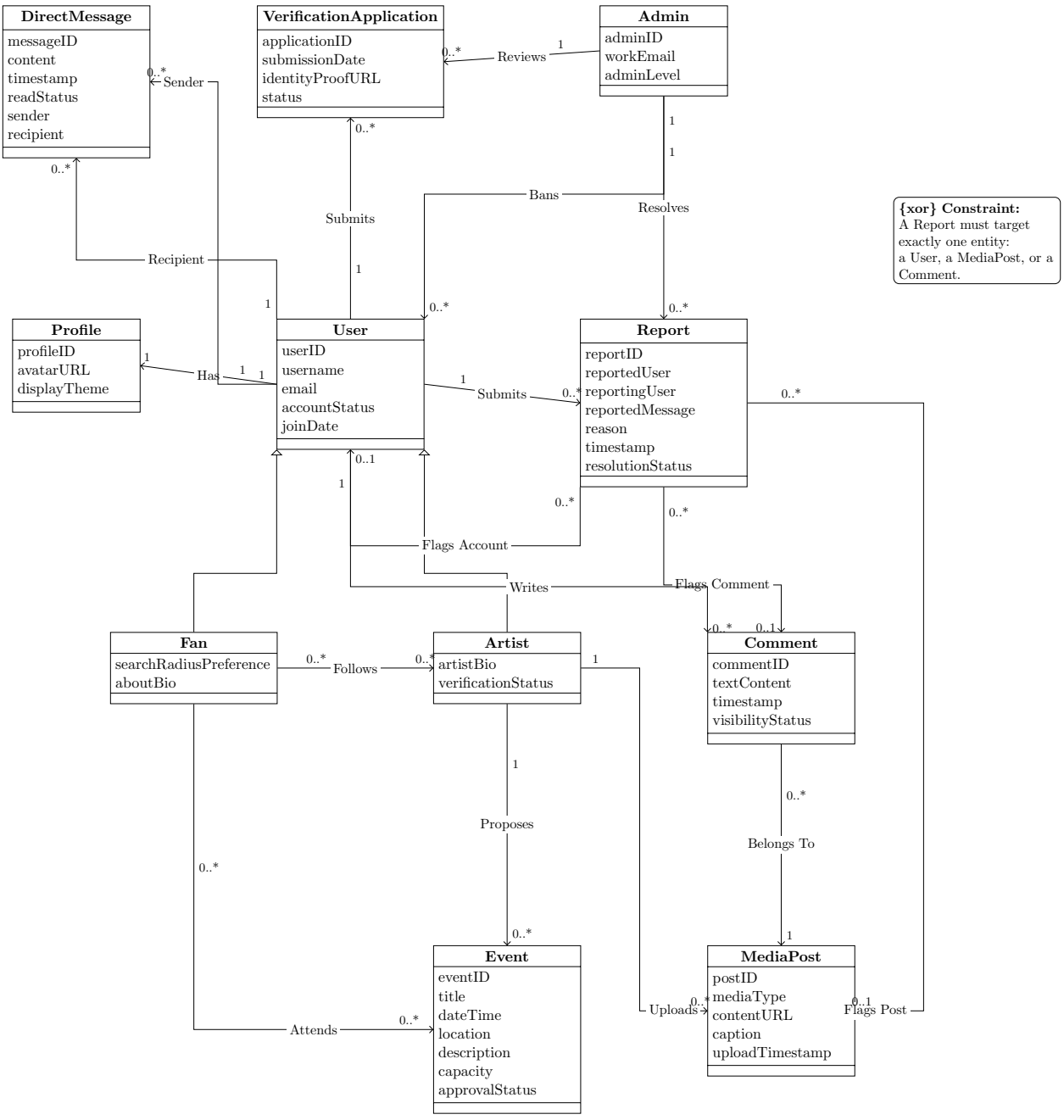
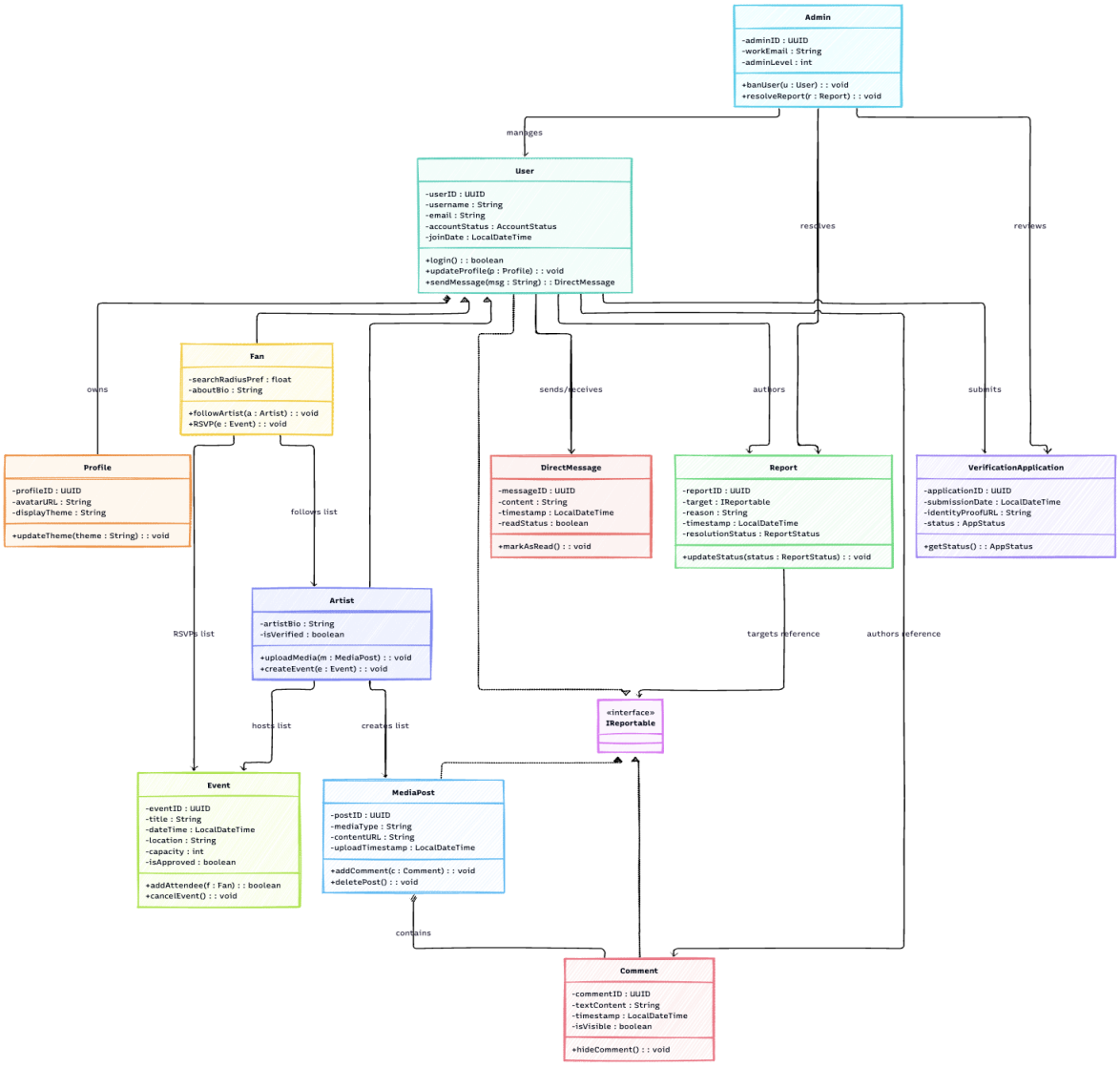


Figure 9.1: System Use Case Diagram

10. Domain Model



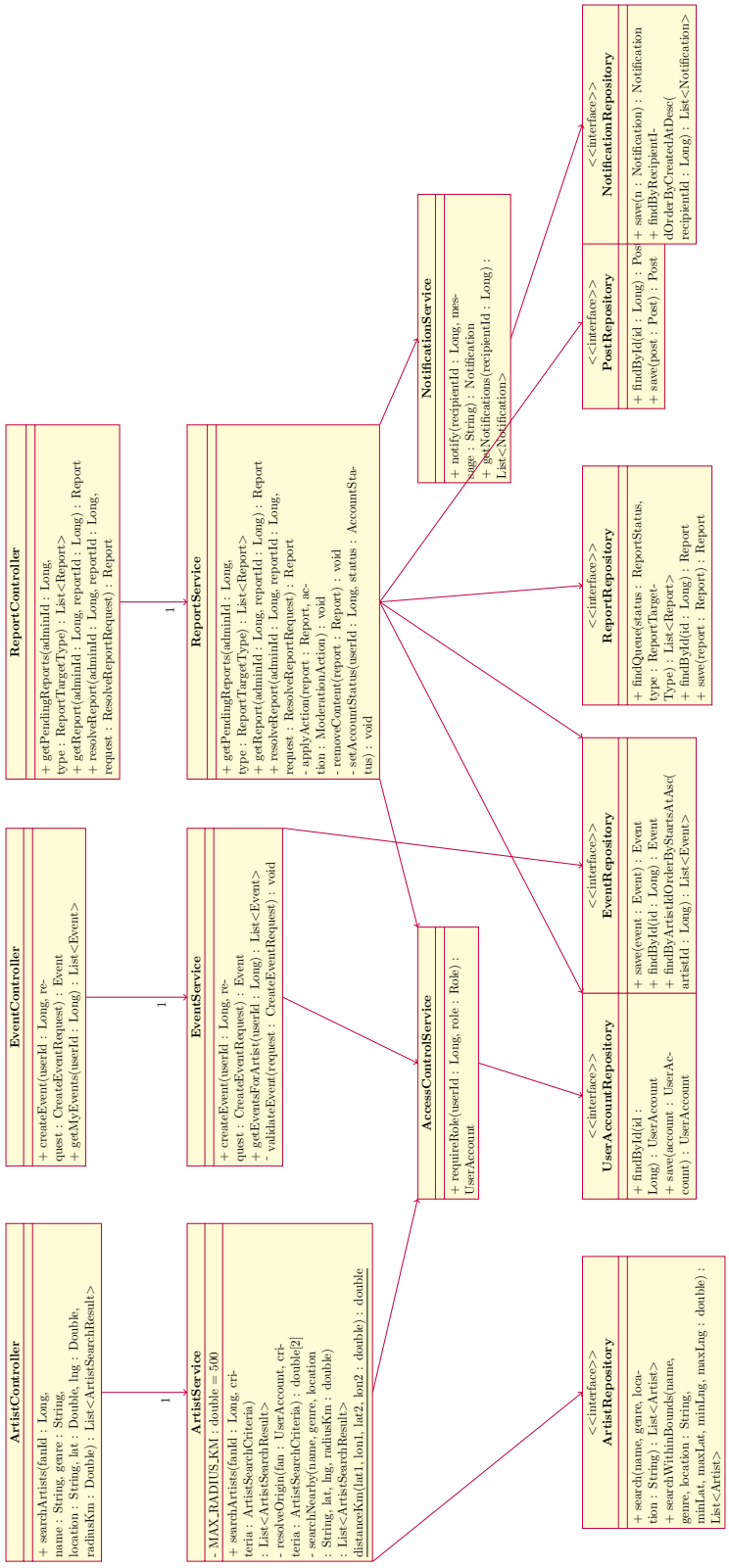
11. Design Class Diagram



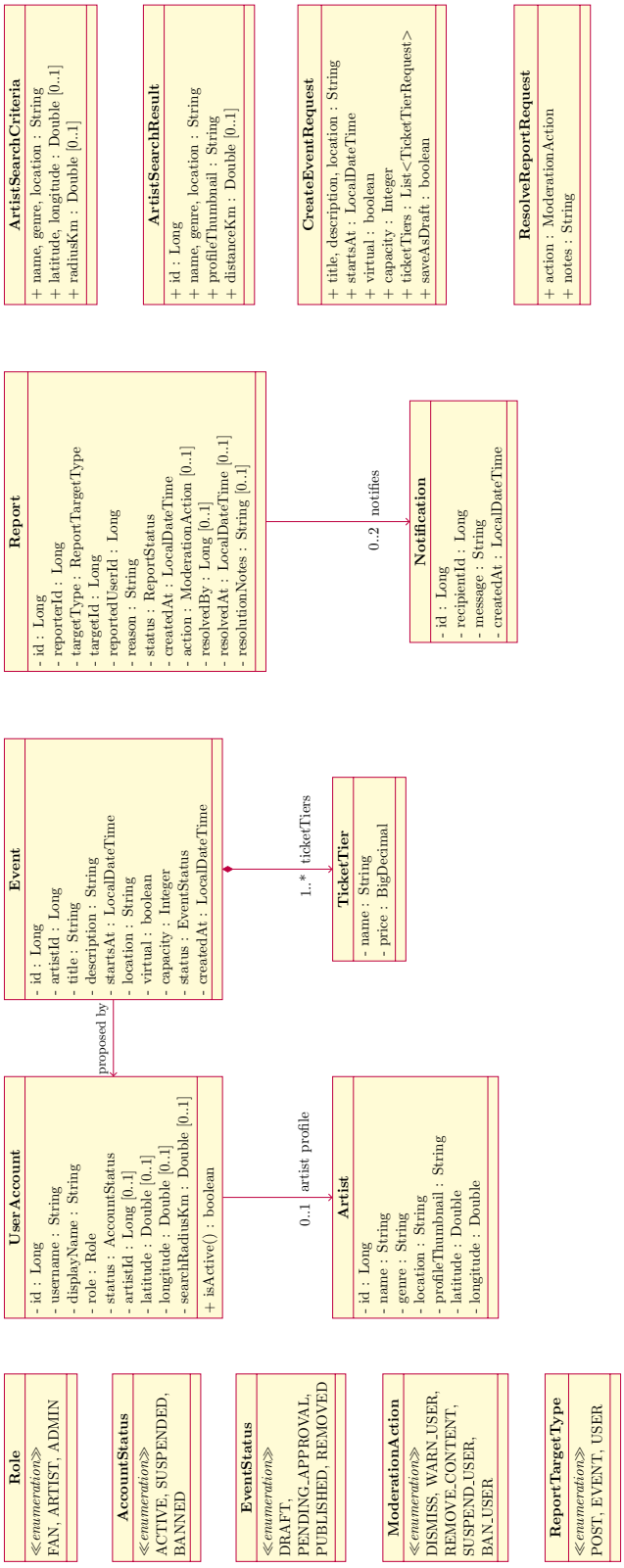
11.1 Design Classes – Search for Artist, Create Event, Review Report

The classes below are the ones used in Alex Garcia's sequence diagrams, with the operations and types from the implementation. Every Fan, Artist, and Admin operation goes through `AccessControlService.requireRole`, which only lets an active account with the required role continue.

Controllers, services, and repositories



Entities, request/response objects, and enumerations



12. Implementation Overview

This section explains how the backend system is implemented at a high level. The ArtistConnect backend is built using a layered, three-tier architecture in Java, dividing responsibilities into distinct packages for data routing, business logic, and database persistence.

12.1 Architecture

The system architecture is strictly organized into major layers, each with a single, well-defined responsibility. This separation of concerns ensures that the application is scalable, testable, and easier to maintain.

- **Controller Layer (`controller/`):** This layer serves as the entry point for the REST API. It handles incoming HTTP requests, extracts parameters and payloads (such as JSON data), delegates processing to the appropriate services, and returns formatted HTTP responses. It includes domain-specific controllers (e.g., `ArtistController`, `EventController`, `ReportController`) and utility classes like `CurrentUser` for session and identity management.
- **Service Layer (`service/`):** The service layer acts as the "brain" of the application, containing all core business logic and workflows. It enforces business rules, verifies permissions, and coordinates data flow between multiple repositories. For example, `EventService` handles the logic of drafting or publishing an event, while `AccessControlService` strictly enforces role-based permissions before any action occurs.
- **Repository Layer (`repository/`):** This data access layer handles all database persistence. It isolates the rest of the application from the underlying database technologies (e.g., SQL queries). By utilizing interfaces (like `EventRepository` or `UserAccountRepository`), it provides clean methods for creating, reading, updating, and deleting domain entities.
- **Domain Model (`model/`):** This package defines the core entities of the application that map to database tables (e.g., `UserAccount`, `Event`, `Post`, `Report`). It also contains critical enumerations (`Role`, `AccountStatus`, `ReportStatus`) that govern application state.
- **Data Transfer Object Layer (`dto/`):** This layer contains strictly typed objects used to pass data between the client and server (e.g., `CreateEventRequest`, `ArtistSearchCriteria`). They act as the public API contract, completely decoupled from the internal database models.

12.2 Key Design Decisions

During the implementation of Iteration II, several important structural choices were made to support the specific functional and non-functional requirements of the ArtistConnect platform:

- **Centralized Access Control:** Rather than checking permissions ad-hoc inside every controller, we implemented a dedicated `AccessControlService`. Because ArtistConnect relies heavily on strict Role-Based Access Control (Admins, Artists, Fans) and account states (Suspended, Banned, Active), funneling all authorization checks through a single service prevents security oversights and reduces code duplication.
- **Separation of Models and DTOs:** We intentionally decoupled our database entities (`model/`) from our API payloads (`dto/`). Using classes like `CreateEventRequest` instead of accepting a raw `Event` object prevents over-posting attacks, hides sensitive internal attributes (like database IDs or password hashes) from the frontend, and allows us to validate incoming data safely before mapping it to a persistent entity.
- **Domain-Driven Service Segregation:** Instead of a monolithic `UserService` handling everything, the logic is highly segregated into bounded contexts (e.g., `FanCommunityService`, `ModerationService`, `ConversationService`, `PostService`). This adheres to the Single Responsibility Principle (SRP) and ensures that, for instance, the complex logic for messaging and notifications does not bloat the core artist discovery logic.
- **Dedicated Moderation Subsystem:** Content safety was identified as a key risk in Iteration I. In response, we designed a dedicated moderation ecosystem (`Report`, `ReportEvidence`, `ReportController`, `ModerationService`). By isolating moderation into its own pipeline, we can more easily integrate future AI-based moderation tools and provide Administrators with clean, auditable queues of user conduct reports.

13. Operation Contracts

Operation: searchArtists(fanId: Long, criteria: ArtistSearchCriteria): List⟨ArtistSearchResult⟩

Cross Reference

UC: Search for Artist – Fan

Preconditions

- A UserAccount instance identified by `fanId` exists with `role` = FAN and `status` = ACTIVE.

Postconditions

- A collection of Artist instances matching every given criterion (name, genre, location) was retrieved.
- If `criteria.radiusKm` was given, only Artist instances within that distance of the search origin (the given coordinates, or else the Fan's saved `latitude`/`longitude`) were retrieved, ordered by distance.
- No instances were created, deleted, or modified (read-only operation).

Operation: resolveReport(adminId: Long, reportId: Long, request: ResolveReportRequest): Report

Cross Reference

UC: Review Report – Admin

Preconditions

- A UserAccount instance identified by `adminId` exists with `role` = ADMIN and `status` = ACTIVE.
- A Report instance identified by `reportId` exists with `status` = PENDING.

Postconditions

- The Report instance's `status` was changed to RESOLVED, and its `action`, `resolvedBy` (the Admin), `resolvedAt`, and `resolutionNotes` were set.
- If `action` = REMOVE_CONTENT, the reported Post's `hidden` was set to true, or the reported Event's `status` was changed to REMOVED.
- If `action` = SUSPEND_USER or BAN_USER, the reported UserAccount's `status` was changed to SUSPENDED or BANNED respectively.
- A new Notification instance was created for the reporting user.
- If `action` $\neq$ DISMISS, a new Notification instance was created for the reported user.

Operation: createEvent(userId: Long, request: CreateEventRequest): Event

Cross Reference

UC: Create Event – Artist

Preconditions

- A UserAccount instance identified by `userId` exists with `role` = ARTIST, `status` = ACTIVE, and a linked Artist profile.

Postconditions

- A new Event instance was created with `title`, `description`, `startsAt`, `location`, `virtual`, and `capacity` set from `request`.
- One TicketTier instance was created for each tier in `request` and associated with the new Event.
- The new Event was associated with the Artist profile of the requesting account.
- The new Event's `status` was set to DRAFT if `request.saveAsDraft` is true, otherwise to PENDING_APPROVAL.

14. Testing

14.1 JUnit Testing Summary

This section covers automated integration tests for each team member's use cases.

The backend test suite uses Spring Boot, MockMvc, and the embedded H2 database to test controllers, services, and repositories against real application state. Tests verify HTTP response codes, response JSON bodies, and database records. Repository exceptions are tested with Mockito to ensure transactions roll back on failure, and concurrent requests are tested to verify that duplicate records are prevented.

The integration tests cover each member's use cases:

- **Alejandro (Alex) Garcia Jr:** Artist discovery search (by text criteria and location radius), event creation with draft and approval statuses, and administrator report queue reviews.
- **Nicholas Nolen:** Event updates with review acknowledgment, fan ban appeals with admin conflict-of-interest checks, and personalized recommendations using weighted interaction scores.
- **William Waweru:** Following and unfollowing artists, fan profile editing with privacy settings, and reporting inappropriate content with evidence retention.
- **Gan-Orgil Gantumur:** Suspending artist accounts, reviewing artist verification applications, and fan comments on artwork.
- **Garrett DeYong:** Creating posts with media attachments, starting direct message conversations between users, and monitoring and managing comments.

14.2 Test Cases Table

Search for Artist – Fan (Alejandro Garcia Jr)

Test Case ID #	**AG-SA-01**
Assigned Tester	Alejandro (Alex) Garcia Jr
Scenario/Condition	Fan searches artists by name, genre, or location criteria.
Test Inputs	GET /api/artists/search?name=Luna from authenticated Fan account (id=1).
Expected Result	HTTP 200 OK. Returns JSON array containing only matching artists (Luna Park).
Actual Result	Observed HTTP 200 OK. Returned exactly 1 artist result matching "Luna Park".
Test Status	Passed

Test Case ID #	**AG-SA-02**
Assigned Tester	Alejandro (Alex) Garcia Jr
Scenario/Condition	Fan submits search request without providing any search criteria (Extension 3a).
Test Inputs	GET /api/artists/search with all search parameters null or empty.
Expected Result	HTTP 400 Bad Request with response body prompting for at least one search criterion.
Actual Result	Observed HTTP 400 Bad Request with validation message requiring criteria.
Test Status	Passed

Test Case ID #	**AG-SA-03**
Assigned Tester	Alejandro (Alex) Garcia Jr
Scenario/Condition	Fan searches within 100 km / 200 km radius of saved home location (Waco, TX).
Test Inputs	GET /api/artists/search?lat=31.5493&lng=-97.1467&radiusKm=150 with Fan account in Waco.
Expected Result	HTTP 200 OK. Returns nearby artists ordered nearest first, including calculated distanceKm.
Actual Result	Observed HTTP 200 OK. Results sorted by ascending distance from Waco coordinates.
Test Status	Passed

Test Case ID #	**AG-SA-04**
Assigned Tester	Alejandro (Alex) Garcia Jr
Scenario/Condition	Fan shares live GPS coordinates in Austin and filters by genre (Extension 3b).
Test Inputs	GET /api/artists/search?lat=30.2672&lng=-97.7431&genre=Indie+Folk&radiusKm=50.
Expected Result	HTTP 200 OK. Returns matching genre artist under 1 km away.
Actual Result	Observed HTTP 200 OK. Returned Luna Park located 0.8 km from shared Austin coordinates.
Test Status	Passed

Test Case ID #	**AG-SA-05**
Assigned Tester	Alejandro (Alex) Garcia Jr
Scenario/Condition	Nearby radius search requested with no shared GPS or saved home coordinates (Extension 3c).
Test Inputs	GET /api/artists/search?radiusKm=50 from Fan account 5 (no location saved, no GPS coordinates).
Expected Result	HTTP 400 Bad Request prompting fan to enable GPS or save a home location.
Actual Result	Observed HTTP 400 Bad Request with location requirement message.
Test Status	Passed

Test Case ID #	**AG-SA-06**
Assigned Tester	Alejandro (Alex) Garcia Jr
Scenario/Condition	Search radius out of permitted platform bounds (0 km or greater than 500 km).
Test Inputs	GET /api/artists/search?lat=30.26&lng=-97.74&radiusKm=0 and radiusKm=600.
Expected Result	HTTP 400 Bad Request indicating radiusKm must be between 1 and 500 km.
Actual Result	Observed HTTP 400 Bad Request for both boundary violation inputs.
Test Status	Passed

Test Case ID #	**AG-SA-07**
Assigned Tester	Alejandro (Alex) Garcia Jr
Scenario/Condition	Role authorization enforcement for artist discovery search (*a).
Test Inputs	Unauthenticated request, or request with Artist (id=2) or Admin (id=3) account.
Expected Result	HTTP 401 Unauthorized for unauthenticated; HTTP 403 Forbidden for non-fan roles.
Actual Result	Observed HTTP 401 and 403 respectively; search endpoint restricted to fans.
Test Status	Passed

Create Event – Artist (Alejandro Garcia Jr)

Test Case ID #	**AG-CE-01**
Assigned Tester	Alejandro (Alex) Garcia Jr
Scenario/Condition	Artist submits a valid event with two ticket tiers.
Test Inputs	POST /api/events with title "Album Release Party", future date, 2 tiers, capacity=200, saveAsDraft=false.
Expected Result	HTTP 201 Created. Event status set to PENDING_APPROVAL with ticket tiers stored.
Actual Result	Observed HTTP 201 Created. Status PENDING_APPROVAL and 2 ticket tiers persisted.
Test Status	Passed

Test Case ID #	**AG-CE-02**
Assigned Tester	Alejandro (Alex) Garcia Jr
Scenario/Condition	Artist saves an event as a draft (Extension 4a).
Test Inputs	POST /api/events with saveAsDraft=true.
Expected Result	HTTP 201 Created. Event status set to DRAFT; listed in artist's private events.
Actual Result	Observed HTTP 201 Created with status DRAFT. Event visible in artist management view.
Test Status	Passed

Test Case ID #	**AG-CE-03**
Assigned Tester	Alejandro (Alex) Garcia Jr
Scenario/Condition	Event creation input validation failures (Extensions 5a, 5b).
Test Inputs	POST /api/events with missing title, past date, negative tier price, zero capacity, or virtual without link.
Expected Result	HTTP 400 Bad Request. No event or ticket tier records persisted in the database.
Actual Result	Observed HTTP 400 Bad Request for all invalid inputs; repository count unchanged.
Test Status	Passed

Test Case ID #	**AG-CE-04**
Assigned Tester	Alejandro (Alex) Garcia Jr
Scenario/Condition	Role authorization enforcement for event creation (*a).
Test Inputs	POST /api/events invoked by unauthenticated caller, Fan account (id=1), or Admin (id=3).
Expected Result	HTTP 401 Unauthorized or HTTP 403 Forbidden. Creation blocked; nothing saved.
Actual Result	Observed HTTP 401 and 403 respectively; only verified Artists permitted to create events.
Test Status	Passed

Test Case ID #	**AG-CE-05**
Assigned Tester	Alejandro (Alex) Garcia Jr
Scenario/Condition	Suspended artist attempts to create an event (*a).
Test Inputs	POST /api/events called by Artist account with isSuspended=true.
Expected Result	HTTP 403 Forbidden. Creation blocked with message indicating account suspension.
Actual Result	Observed HTTP 403 Forbidden. Suspended artist prevented from publishing events.
Test Status	Passed

Review Report – Admin (Alejandro Garcia Jr)

Test Case ID #	**AG-RR-01**
Assigned Tester	Alejandro (Alex) Garcia Jr
Scenario/Condition	Admin opens the pending report queue with optional target type filtering.
Test Inputs	GET /api/reports/pending?targetType=POST with Admin credentials (adminId=3).
Expected Result	HTTP 200 OK. Returns pending reports ordered oldest first by createdAt.
Actual Result	Observed HTTP 200 OK. Queue returned with pending reports sorted in ascending order.
Test Status	Passed

Test Case ID #	**AG-RR-02**
Assigned Tester	Alejandro (Alex) Garcia Jr
Scenario/Condition	Non-admin user attempts to access or resolve the report queue (*a).
Test Inputs	GET /api/reports/pending and POST /api/reports/{id}/resolve called by Fan or Artist.
Expected Result	HTTP 401 Unauthorized or HTTP 403 Forbidden. No report states modified.
Actual Result	Observed HTTP 403 Forbidden. Access strictly limited to system administrators.
Test Status	Passed

Test Case ID #	**AG-RR-03**
Assigned Tester	Alejandro (Alex) Garcia Jr
Scenario/Condition	Admin removes a reported post or event upon report resolution.
Test Inputs	POST /api/reports/{id}/resolve with action=REMOVE_CONTENT and resolution notes.
Expected Result	HTTP 200 OK. Target post marked hidden / event REMOVED; both reporter and content owner notified.
Actual Result	Observed HTTP 200 OK. Content hidden; notification dispatched to both users.
Test Status	Passed

Test Case ID #	**AG-RR-04**
Assigned Tester	Alejandro (Alex) Garcia Jr
Scenario/Condition	Admin dismisses a report (Extension 5a).
Test Inputs	POST /api/reports/{id}/resolve with action=DISMISS.
Expected Result	HTTP 200 OK. Report marked RESOLVED with action DISMISS; only reporter notified; content remains active.
Actual Result	Observed HTTP 200 OK. Content unaltered; reporter received dismissal notification.
Test Status	Passed

Test Case ID #	**AG-RR-05**
Assigned Tester	Alejandro (Alex) Garcia Jr
Scenario/Condition	Admin bans the reported user account (Extension 5b).
Test Inputs	POST /api/reports/{id}/resolve with action=BAN_USER.
Expected Result	HTTP 200 OK. Target UserAccount status set to BANNED; user blocked from platform actions.
Actual Result	Observed HTTP 200 OK. UserAccount status persisted as BANNED in database.
Test Status	Passed

Test Case ID #	**AG-RR-06**
Assigned Tester	Alejandro (Alex) Garcia Jr
Scenario/Condition	Admin attempts to resolve an already resolved report (Extension 3a).
Test Inputs	POST /api/reports/{id}/resolve targeting a report with status already RESOLVED.
Expected Result	HTTP 409 Conflict. Duplicate resolution rejected; original resolution preserved.
Actual Result	Observed HTTP 409 Conflict with message indicating report has already been resolved.
Test Status	Passed

Test Case ID #	**AG-RR-07**
Assigned Tester	Alejandro (Alex) Garcia Jr
Scenario/Condition	Invalid report resolution inputs (Extension 5c).
Test Inputs	POST /api/reports/{id}/resolve with REMOVE_CONTENT on USER report, missing action, or unknown ID.
Expected Result	HTTP 400 Bad Request for invalid/missing action; HTTP 404 for unknown report ID.
Actual Result	Observed HTTP 400 and HTTP 404 respectively; database state unmodified.
Test Status	Passed

Update Event – Artist (Nicholas Nolen)

Test Case ID #	**NN-UE-01**
Assigned Tester	Nicholas Nolen
Scenario/Condition	Artist updates event details with review acknowledgment confirmed.
Test Inputs	PUT /api/events/{id} with updated title, location, capacity, and reviewedAcknowledgment=true.
Expected Result	HTTP 200 OK. Event details updated in database; updatedAt timestamp set to current time.
Actual Result	Observed HTTP 200 OK with updated title and capacity. Database query confirmed reviewedAcknowledgment=true.
Test Status	Passed

Test Case ID #	**NN-UE-02**
Assigned Tester	Nicholas Nolen
Scenario/Condition	Reject event update when reviewedAcknowledgment is false (Extension 6a).
Test Inputs	PUT /api/events/{id} with valid event fields but reviewedAcknowledgment=false.
Expected Result	HTTP 400 Bad Request / IllegalStateException with message "Artist must acknowledge they have reviewed the details".
Actual Result	Exception caught: IllegalStateException with required acknowledgment message; no updates saved.
Test Status	Passed

Test Case ID #	**NN-UE-03**
Assigned Tester	Nicholas Nolen
Scenario/Condition	Reject event update on empty title or invalid fields (Extension 5a).
Test Inputs	PUT /api/events/{id} with empty string title "" and reviewedAcknowledgment=true.
Expected Result	HTTP 400 Bad Request / IllegalArgumentException with message "Event title cannot be empty".
Actual Result	Exception caught: IllegalArgumentException indicating empty title violation; database unchanged.
Test Status	Passed

Test Case ID #	**NN-UE-04**
Assigned Tester	Nicholas Nolen
Scenario/Condition	Reject event update when requesting artist ID does not match event owner.
Test Inputs	PUT /api/events/{id} with artistId=999 targeting an event owned by artistId=10.
Expected Result	HTTP 403 Forbidden / IllegalArgumentException with message "Unauthorized to edit this event".
Actual Result	Exception caught: IllegalArgumentException with unauthorized message; edit rejected.
Test Status	Passed

Unban Fan – Administrator (Nicholas Nolen)

Test Case ID #	**NN-UF-01**
Assigned Tester	Nicholas Nolen
Scenario/Condition	Retrieve open appeals filtered to enforce conflict-of-interest policy.
Test Inputs	GET /api/moderation/appeals/open?adminId=50 (banning admin) vs ?adminId=60 (neutral admin).
Expected Result	HTTP 200 OK. Returns 0 appeals for admin 50 (filtered out) and 1 appeal for admin 60.
Actual Result	Observed HTTP 200 OK. Length 0 for banning admin 50; length 1 with matching appeal ID for admin 60.
Test Status	Passed

Test Case ID #	**NN-UF-02**
Assigned Tester	Nicholas Nolen
Scenario/Condition	Retrieve appeal details including fan reasoning and AI ToS evaluation.
Test Inputs	GET /api/moderation/appeals/{id} targeting a valid open appeal record.
Expected Result	HTTP 200 OK. Response body includes fan reason, original report material, and AI recommendation.
Actual Result	Observed HTTP 200 OK. Response contained appeal reason and AI recommendation text.
Test Status	Passed

Test Case ID #	**NN-UF-03**
Assigned Tester	Nicholas Nolen
Scenario/Condition	Successfully unban a fan when appeal is approved by neutral administrator.
Test Inputs	POST /api/moderation/appeals/{id}/unban?adminId=60&notes=Restoring+account.
Expected Result	HTTP 200 OK. Appeal status set to APPROVED; reviewingAdminId=60. Fan entity updated: banned=false, canLogin=true.
Actual Result	Observed HTTP 200 OK with status APPROVED. GET /api/moderation/fans/{id} verified banned=false, canLogin=true.
Test Status	Passed

Test Case ID #	**NN-UF-04**
Assigned Tester	Nicholas Nolen
Scenario/Condition	Reject unban review attempt by the same administrator who issued the ban.
Test Inputs	POST /api/moderation/appeals/{id}/unban?adminId=50 (admin who issued the original ban).
Expected Result	HTTP 400 Bad Request / IllegalArgumentException: "A different admin than the banning admin must review the appeal".
Actual Result	Exception caught: IllegalArgumentException with conflict of interest violation message.
Test Status	Passed

Test Case ID #	**NN-UF-05**
Assigned Tester	Nicholas Nolen
Scenario/Condition	Invalidate frivolous appeal and revoke fan appeal privileges.
Test Inputs	POST /api/moderation/appeals/{id}/invalidate?adminId=60&notes=Frivolous+appeal.
Expected Result	HTTP 200 OK. Appeal status set to INVALID. Fan entity updated: banned=true, canLogin=false, canAppeal=false.
Actual Result	Observed HTTP 200 OK with status INVALID. Fan record verified: banned=true, canAppeal=false.
Test Status	Passed

Receive Recommendation – Fan (Nicholas Nolen)

Test Case ID #	**NN-RR-01**
Assigned Tester	Nicholas Nolen
Scenario/Condition	Cold-start discovery recommendation for new fan with no interactions (Extension 1a).
Test Inputs	GET /api/recommendations/fan/999 for newly registered fan account.
Expected Result	HTTP 200 OK. hasInteractions=false, informative starter discovery message, discovery artists populated.
Actual Result	Observed HTTP 200 OK with hasInteractions=false and populated starter discovery recommendations.
Test Status	Passed

Test Case ID #	**NN-RR-02**
Assigned Tester	Nicholas Nolen
Scenario/Condition	Active fan receives weighted genre affinity scores and ranked recommendations.
Test Inputs	Record FOLLOW (weight 5.0) and LIKE (3.0) on Indie Folk artist; VIEW (1.0) on Jazz. Request GET /api/recommendations/fan/{id}.
Expected Result	HTTP 200 OK. hasInteractions=true. Genre affinity Indie Folk ¿ Jazz. Top recommendation matches Indie Folk (¿ 80% match).
Actual Result	Observed HTTP 200 OK with hasInteractions=true. Luna Park recommended first with match percentage ¿ 80% and reason.
Test Status	Passed

Test Case ID #	**NN-RR-03**
Assigned Tester	Nicholas Nolen
Scenario/Condition	Record user interaction endpoint persists weighted taste interaction.
Test Inputs	POST /api/recommendations/interactions?fanId=888&artistId=1&interactionType=FOLLOW.
Expected Result	HTTP 200 OK. UserInteraction saved with interactionType="FOLLOW" and weight 5.0.
Actual Result	Observed HTTP 200 OK. UserInteraction persisted with weight 5.0. Follow-up recommendation reflected updated preference.
Test Status	Passed

Follow Artist – Fan (William Waweru)

Test Case ID #	**WW-FA-01**
Assigned Tester	William Waweru
Scenario/Condition	Follow an active artist and retrieve the relationship; then unfollow.
Test Inputs	Active fan account 1; active artist 1 (Luna Park). Send PUT to the follow endpoint, GET the following list and own profile, then DELETE the follow.
Expected Result	PUT returns HTTP 200 with following=true and followerCount=1. Both lists contain artist 1. DELETE returns 204; the next follow-state read reports followerCount=0.
Actual Result	Observed HTTP 200 and one follower; Luna Park appeared in both lists. Unfollow returned 204 and the count became 0.
Test Status	Passed

Test Case ID #	**WW-FA-02**
Assigned Tester	William Waweru
Scenario/Condition	Repeated and concurrent follow requests must not create duplicate relationships.
Test Inputs	Send the same follow request twice for fan 1 / artist 1. In a separate test, send four simultaneous PUT requests for fan 5 / artist 6.
Expected Result	All requests return HTTP 200. Each fan/artist pair has exactly one follow row and one FOLLOW interaction. Repeated requests do not increase the follower count for the same fan.
Actual Result	Both repeated requests and all four concurrent requests returned 200. Database assertions found one relationship and one FOLLOW interaction per tested pair.
Test Status	Passed

Follow Artist – Fan (continued)

Test Case ID #	**WW-FA-03**
Assigned Tester	William Waweru
Scenario/Condition	Reject a suspended artist, a banned artist account, and an unknown artist.
Test Inputs	Fan 1 attempts to follow artist 1 with its suspension flag set; retries after clearing that flag and banning the linked artist account; also requests artist 9999.
Expected Result	Suspended and banned cases each return HTTP 409. The unknown artist returns 404. No follow relationship is stored for any rejected request.
Actual Result	Observed 409 for each unavailable-artist case and 404 for artist 9999. Follow repository count remained zero.
Test Status	Passed

Test Case ID #	**WW-FA-04**
Assigned Tester	William Waweru
Scenario/Condition	Enforce fan authorization and keep different fans' following lists independent.
Test Inputs	Follow artist 1 with no acting-account header and then artist account 2. Separately, fan 1 follows artist 1; inspect fan 5's list before fan 5 follows the same artist.
Expected Result	Unauthenticated request returns HTTP 401; artist-role request returns 403. Fan 5 initially sees an empty list. After both fans follow, the shared artist count is exactly 2.
Actual Result	Observed 401 and 403. Fan 5's list was initially empty; its own follow returned followerCount=2 without duplicating either fan's relationship.
Test Status	Passed

Manage Fan Profile – Fan (William Waweru)

Test Case ID #	**WW-MFP-01**
Assigned Tester	William Waweru
Scenario/Condition	Save valid profile text and a valid uploaded profile image.
Test Inputs	Fan 1 saves display name New Fan Name, biography Love independent art, interests Jazz and art, and valid privacy values. A separate request uploads a generated 2-by-2-pixel PNG as a data URL.
Expected Result	Each valid update returns HTTP 200. Subsequent profile reads return the saved text and exactly the uploaded PNG data URL. Publicly enabled interests and image are visible to fan 5.
Actual Result	Observed 200 for valid updates. Saved text and image matched the submitted values; fan 5 retrieved the enabled public fields.
Test Status	Passed

Test Case ID #	**WW-MFP-02**
Assigned Tester	William Waweru
Scenario/Condition	Apply field-level and whole-profile privacy on the server.
Test Inputs	Fan 1 saves a public profile with showInterests=false and showFollowing=false. Fan 5 requests that profile. Fan 1 then sets publicProfile=false; repeat reads as fan 5 and as the owner.
Expected Result	For the public profile, fan 5 receives HTTP 200 without interests or supportedArtists values. Once private, fan 5 receives 404 while the owner still receives 200 with their full profile.
Actual Result	Public response omitted the hidden fields. After the privacy change, the other fan received 404 and the owner retained 200 access.
Test Status	Passed

Manage Fan Profile – Fan (continued)

Test Case ID #	**WW-MFP-03**
Assigned Tester	William Waweru
Scenario/Condition	Reject invalid content, malformed images, and stale edits without replacing valid data.
Test Inputs	After saving Saved Name / Saved bio, submit a blank name and then a biography containing script markup. Separately submit a malformed PNG data URL and a profile update carrying an outdated version.
Expected Result	Blank-name, prohibited-content, and malformed-image requests return HTTP 400. The blank-name response identifies displayName. A stale version returns 409. Rejected requests preserve the previous stored profile.
Actual Result	Observed 400 with a displayName field error, 400 for script markup and malformed PNG data, and 409 for the stale edit. Stored Saved Name / Saved bio remained unchanged after the invalid text requests.
Test Status	Passed

Test Case ID #	**WW-MFP-04**
Assigned Tester	William Waweru
Scenario/Condition	Preserve both profile and account data when a database write fails.
Test Inputs	Save Persisted Name / Persisted biography for fan 1. Configure the profile repository spy to throw a database write exception, then attempt to save Must roll back / Must not persist.
Expected Result	The update returns HTTP 503. The transaction rolls back: the profile biography remains Persisted biography and the account display name remains Persisted Name.
Actual Result	Observed HTTP 503. Follow-up database reads confirmed the previous biography and account display name; neither attempted replacement was committed.
Test Status	Passed

Report Inappropriate Content – Fan (William Waweru)

Test Case ID #	**WW-RIC-01**
Assigned Tester	William Waweru
Scenario/Condition	Submit a valid report and make it available to the existing administrator queue.
Test Inputs	Fan 1 reports published EVENT 2 with reason Spam and details Please review this context. Admin account 3 retrieves the pending report queue.
Expected Result	Submission returns HTTP 201 with status=PENDING and reportedUserId=2. The saved snapshot includes Austin Acoustic Night. The seeded three-report admin queue increases to four.
Actual Result	Observed 201, PENDING, owner account 2, and the expected event snapshot. The admin queue contained four reports, including the new submission.
Test Status	Passed

Test Case ID #	**WW-RIC-02**
Assigned Tester	William Waweru
Scenario/Condition	Reject missing reasons, unavailable content, and callers without the fan role.
Test Inputs	Submit a whitespace-only reason for EVENT 2; report nonexistent POST 9999; report unpublished EVENT 3; and attempt reporting as artist account 2.
Expected Result	Missing reason returns HTTP 400. Missing post and unpublished event each return 404. The artist-role request returns 403. No rejected request adds a report; the seeded count stays at 3.
Actual Result	Observed 400, 404, 404, and 403 respectively. Database report count remained three.
Test Status	Passed

Report Inappropriate Content – Fan (continued)

Test Case ID #	**WW-RIC-03**
Assigned Tester	William Waweru
Scenario/Condition	Prevent duplicate pending reports, including simultaneous submissions.
Test Inputs	Fan 1 submits EVENT 2 twice. In a separate test, fan 5 sends four simultaneous reports for EVENT 2 with reason Spam.
Expected Result	The first sequential request returns HTTP 201 and the duplicate returns 409. The concurrent set produces exactly one 201 and three 409 responses, leaving only one new pending report for that fan/target.
Actual Result	Observed 201 then 409 for the sequential pair. Concurrent results were exactly 201, 409, 409, 409; duplicate pending reports were not created.
Test Status	Passed

Test Case ID #	**WW-RIC-04**
Assigned Tester	William Waweru
Scenario/Condition	Allow reporting deleted content only with evidence captured for the same fan and target.
Test Inputs	Fan 5 captures server evidence for POST 1000; delete that post. Submit without evidence, try the captured ID as fan 1 against EVENT 2, then submit as fan 5 for POST 1000 using the matching evidence.
Expected Result	Without evidence the deleted-post request returns HTTP 404. Evidence with the wrong fan/target returns 404. Matching retained evidence returns 201 and preserves the snapshot text New single out Friday.
Actual Result	Observed 404 for missing evidence and mismatched ownership/target. Matching evidence returned 201; the saved report retained the original post text.
Test Status	Passed

Suspend Artist Account – Administrator (Gan-Orgil Gantumur)

Test Case ID #	**GG-SA-01**
Assigned Tester	Gan-Orgil Gantumur
Scenario/Condition	Admin successfully suspends an active Artist account.
Test Inputs	PUT /api/moderation/artists/{id}/suspend targeting a valid, unsuspended Artist.
Expected Result	HTTP 200 OK. The JSON response shows suspended=true, and the database reflects the updated state.
Actual Result	Observed HTTP 200 OK with suspended=true. Database query verified the isSuspended flag was persisted as true.
Test Status	Passed

Comment on Artwork – Fan (Gan-Orgil Gantumur)

Test Case ID #	**GG-CA-01**
Assigned Tester	Gan-Orgil Gantumur
Scenario/Condition	Fan successfully comments on an Artist's post, generating a weighted taste profile interaction.
Test Inputs	POST /api/posts/{postId}/comments with fanId, artistId, genre parameters, and a text body payload.
Expected Result	HTTP 200 OK. The system saves a new UserInteraction entity with type "COMMENT" and a default algorithmic weight of 3.0.
Actual Result	Observed HTTP 200 OK. The UserInteraction table incremented by 1, correctly asserting a weight of 3.0 and automated timestamp creation.
Test Status	Passed

Approve Artist Verification Application – Administrator (Gan-Orgil Gantumur)

Test Case ID #	**GG-AV-01**
Assigned Tester	Gan-Orgil Gantumur
Scenario/Condition	Admin approves a pending Artist Verification application.
Test Inputs	PUT /api/moderation/applications/{id}/approve targeting an application with status PENDING.
Expected Result	HTTP 200 OK. Application status updates to APPROVED. A new Artist entity is successfully provisioned and verifiable in the ArtistRepository.
Actual Result	Observed HTTP 200 OK. Application status changed to APPROVED. Baseline artist count incremented by 1, and the provisioned artist matched the submitted application details.
Test Status	Passed

Test Case ID #	**GG-AV-02**
Assigned Tester	Gan-Orgil Gantumur
Scenario/Condition	Admin rejects a pending Artist Verification application.
Test Inputs	PUT /api/moderation/applications/{id}/reject targeting an application with status PENDING.
Expected Result	HTTP 200 OK. Application status updates to REJECTED. No new Artist entity is provisioned in the ArtistRepository.
Actual Result	Observed HTTP 200 OK with status REJECTED. The Artist table count remained unchanged against the baseline, verifying no ghost profiles were created.
Test Status	Passed

Make a Post – Artist (Garrett DeYong)

Test Case ID #	**GD-MP-01**
Assigned Tester	Garrett DeYong
Scenario/Condition	Artist creates a new post with text and attached media files.
Test Inputs	POST /api/posts with PostWrapper containing posterId=2, postText="Studio session preview", isPublic=true, and attached PNG PostFile.
Expected Result	HTTP 200 OK. Stored PostWrapper returned with assigned post ID, posterId, and attached PostFile records.
Actual Result	Observed HTTP 200 OK. Post persisted in repository with attached media file correctly associated.
Test Status	Passed

Test Case ID #	**GD-MP-02**
Assigned Tester	Garrett DeYong
Scenario/Condition	Reject post creation when required content is missing (Extension 4a).
Test Inputs	POST /api/posts with empty postText and empty files list.
Expected Result	HTTP 400 Bad Request or validation error. No post or file records persisted in the database.
Actual Result	Observed HTTP 400 Bad Request. Post repository count remained unchanged.
Test Status	Passed

Test Case ID #	**GD-MP-03**
Assigned Tester	Garrett DeYong
Scenario/Condition	Retrieve all published posts by artist profile ID.
Test Inputs	GET /api/posts/byuserid/2 targeting an artist with existing published posts.
Expected Result	HTTP 200 OK. Returns JSON array of Post entities matching posterId=2.
Actual Result	Observed HTTP 200 OK. Returned list of posts authored by artist 2.
Test Status	Passed

Message Another User (Garrett DeYong)

Test Case ID #	**GD-MU-01**
Assigned Tester	Garrett DeYong
Scenario/Condition	Begin a new direct messaging conversation between two users.
Test Inputs	POST /api/conversations with UserWrapper containing userId1=1 (Fan) and userId2=2 (Artist).
Expected Result	HTTP 200 OK. New Conversation entity created with distinct conversationId and timestamps.
Actual Result	Observed HTTP 200 OK. Returned Conversation with initialized conversationId and empty message thread.
Test Status	Passed

Test Case ID #	**GD-MU-02**
Assigned Tester	Garrett DeYong
Scenario/Condition	Send and append a direct message to an active conversation thread.
Test Inputs	POST /api/conversations/messages with MessageWrapper containing conversationId, senderId=1, recipientId=2, and text content.
Expected Result	HTTP 200 OK. Stored Message returned with timestamp and read=false, appended to conversation.
Actual Result	Observed HTTP 200 OK. Message persisted with correct senderId, recipientId, content, and read=false.
Test Status	Passed

Test Case ID #	**GD-MU-03**
Assigned Tester	Garrett DeYong
Scenario/Condition	Retrieve conversation history by conversation ID.
Test Inputs	GET /api/conversations/byconversationid/{id} for existing active conversation.
Expected Result	HTTP 200 OK. Returns Conversation entity with chronological list of message exchanges.
Actual Result	Observed HTTP 200 OK. Response contained full message history matching conversation ID.
Test Status	Passed

Monitor Comments – Artist (Garrett DeYong)

Test Case ID #	**GD-MC-01**
Assigned Tester	Garrett DeYong
Scenario/Condition	Retrieve post details and comment preferences for artist monitoring.
Test Inputs	GET /api/posts/bypostid/{postId} by authoring artist.
Expected Result	HTTP 200 OK. Returns post details including commentsAllowed setting.
Actual Result	Observed HTTP 200 OK with post details and commentsAllowed=true.
Test Status	Passed

Test Case ID #	**GD-MC-02**
Assigned Tester	Garrett DeYong
Scenario/Condition	Fan comment on post artwork generates weighted taste profile interaction.
Test Inputs	POST /api/posts/{postId}/comments?fanId=5&artistId=2&genre=Indie+Rock with text payload "Stunning artwork!".
Expected Result	HTTP 200 OK. Saves UserInteraction with type "COMMENT" and algorithmic weight 3.0.
Actual Result	Observed HTTP 200 OK. UserInteraction count incremented by 1, with weight 3.0 and timestamp asserted.
Test Status	Passed

Test Case ID #	**GD-MC-03**
Assigned Tester	Garrett DeYong
Scenario/Condition	Hide offensive comment from public view without deleting from audit log.
Test Inputs	Update comment visibility setting hidden=true on flagged comment.
Expected Result	HTTP 200 OK. Comment hidden from public view while retained in database for admin audit.
Actual Result	Observed HTTP 200 OK. Flagged comment omitted from public retrieval but retained in database.
Test Status	Passed

15. Setup / Installation Guide

This guide explains how to reproduce ArtistConnect on a local computer, start both application tiers, and run the automated checks. The commands use Windows PowerShell unless a macOS/Linux alternative is given. The backend uses Spring Boot and an embedded H2 database; the frontend uses Next.js.

15.1 Required Tools

Install the following tools before starting:

Tool	Requirement
Git	Required to clone the repository and select the project branch.
Java Development Kit	JDK 21, including the Java compiler (`javac`). A Java runtime alone is insufficient.
Node.js and npm	An LTS version of Node.js meeting the frontend's minimum of 20.9.0, with npm installed.
Web browser	Required to open the application. Automated browser tests additionally require Playwright and Chromium.

Maven does not require a separate installation: the repository includes a Maven wrapper. A separate database server is also unnecessary because the backend creates H2 and loads its sample records automatically. Internet access is required for initial dependency downloads. The current backend uses Spring Boot `4.1.2-SNAPSHOT`, with its snapshot repository configured in the Maven build.

Installation sources are Git, Eclipse Temurin JDK 21, and Node.js.

15.2 Environment Configuration

Set `JAVA_HOME` to the JDK installation directory, not its `bin` subdirectory. Add the JDK's `bin` directory to `PATH`.

On Windows, use the installer's environment-variable options when available. Otherwise, open **Edit the system environment variables**, select **Environment Variables**, create or update `JAVA_HOME`, and add `%JAVA_HOME%\bin` to `Path`. On macOS/Linux, configure the equivalent variables through the JDK installation or shell environment. Open a new terminal after making changes.

Verify the tools before continuing:

```
git --version
java -version
javac -version
node --version
npm --version
```

Both Java commands should report version 21. Node.js must be at least 20.9.0. If a command is not recognized, correct its installation or environment configuration before proceeding.

No environment file is needed for the default local setup. The frontend expects the backend at `http://localhost:8080`. If a different address is necessary, create `frontend/.env.local`, set `NEXT_PUBLIC_API_URL` to that address, and restart the frontend. A production frontend build must be rebuilt after changing this value. The backend currently permits the frontend origin `http://localhost:3000`; changing that origin also requires updating its CORS configuration.

15.3 Download the Project

Open a terminal in the directory where the project should be stored:

```
git clone https://github.com/AGarciaJr/CSI5324-IndieDevs.git
cd CSI5324-IndieDevs
git switch william
git branch --show-current
```

The final command should print `william`. If the repository is already available locally, use that checkout and confirm the intended branch instead of cloning again.

The `backend` directory contains the Spring Boot application, Maven wrapper, and JUnit tests. The `frontend` directory contains the Next.js application and browser-test scripts. Run Maven commands inside `backend` and npm commands inside `frontend`.

15.4 Start the Backend

Open **Terminal 1** in the repository root. On Windows PowerShell, run:

```
cd backend
.\mvnw.cmd spring-boot:run
```

On macOS/Linux, use:

```
cd backend
./mvnw spring-boot:run
```

The first run may take several minutes while Maven downloads its dependencies. Wait until the log reports that Tomcat started on port 8080 and that `BackendApplication` started. Leave this terminal running.

Verify startup by opening `http://localhost:8080/api/artists`. A JSON list of sample artists should appear. The backend root URL is not the application's frontend page.

The default database requires no manual schema creation. For optional inspection, open `http://localhost:8080/h2-console`, use JDBC URL `jdbc:h2:mem:testdb`, driver `org.h2.Driver`, user name `sa`, and a blank password.

15.5 Install and Start the Frontend

Open **Terminal 2** in the repository root:

```
cd frontend
npm ci
npm run dev
```

The `npm ci` command installs the versions recorded in the frontend lockfile. Wait for Next.js to report that it is ready, then open `http://localhost:3000`. Keep both terminals running: the frontend uses port 3000 and communicates with the backend on port 8080. The initial fan view should show the artist-discovery search controls, with the demo-account switcher beneath the logo.

Use `localhost:3000` for this setup so that the browser origin matches the backend's CORS configuration. Press **Ctrl+C** in a server's terminal to stop it.

To run the frontend as a production build instead, stop its development server and execute these commands from `frontend`, while keeping the backend running:

```
npm run build
npm run start
```

A successful build is required before `npm run start`. The browser address remains `http://localhost:3000`.

15.6 Run the Automated Checks

15.6.1 Backend JUnit Tests

The backend tests start their own Spring Boot test context and H2 database; the application servers do not need to be running. From the repository root, use Windows PowerShell:

```
cd backend
.\mvnw.cmd test
```

On macOS/Linux, replace the second command with `./mvnw test`. A successful run ends with **BUILD SUCCESS** and zero test failures or errors. Detailed reports are written to `backend/target/surefire-reports/`.

To run only the fan-community tests, execute the following from `backend`:

```
.\mvnw.cmd "-Dtest=FanCommunityTest" test
```

On macOS/Linux, use `./mvnw` instead of `.\mvnw.cmd`.

15.6.2 Frontend Checks

From the repository root, open another terminal and run:

```
cd frontend
npm run lint
npx tsc --noEmit
npm run build
```

These commands check lint rules, TypeScript types, and the production build. There is currently no separate `npm test` script. Two existing hook-dependency lint warnings and the Next.js multiple-lockfile warning should be distinguished from errors.

15.6.3 Browser Integration Tests

Keep the backend and frontend running at their default addresses. In a third terminal, from the repository root, install the optional test tools:

```
cd frontend
npm install --no-save --package-lock=false playwright
npx playwright install chromium
```

This local installation does not change the package manifest or lockfile. A subsequent `npm ci` removes the extra Playwright package, so reinstall it when necessary. If Linux reports missing browser libraries, use `npx playwright install --with-deps chromium`; installation of system libraries may require administrator privileges.

Restart the backend and wait for startup to complete before each script so that the expected sample records are available. From `frontend`, run:

```
node tests/presentation-regression.cjs
```

This checks original demo accounts, search, navigation, existing team screens, and the added fan/admin tabs on desktop and mobile. It does not modify backend records.

After another backend restart, run:

```
node tests/fan-community-smoke.cjs
```

This checks live follow persistence, account isolation, profile validation/privacy, reporting, duplicate protection, administrator resolution, and mobile layout. It changes local demo profile, follow, and report data. Restart the backend again when the original samples are needed for a presentation.

Both scripts print a **PASS** message on success and return a nonzero exit code on failure. Screenshots are saved in `frontend/test-results/`.

15.7 Troubleshooting and Resetting Sample Data

Java or Maven cannot find the JDK.
Verify `JAVA_HOME`, `PATH`, `java -version`, and `javac -version`. An unsupported-release error usually means that an older JDK is active.

PowerShell blocks npm or npx scripts.
Use `npm.cmd` or `npx.cmd`, for example `npm.cmd ci` and `npm.cmd run dev`.

The Maven wrapper reports permission denied on macOS/Linux.
From `backend`, run `chmod +x mvnw` and retry the wrapper command.

Dependencies cannot be downloaded.
Check the internet connection and proxy settings. Maven needs Maven Central and the configured Spring snapshot repository; npm and Playwright also need access to their download services.

Port 8080 or 3000 is already occupied.
Stop the previous server with Ctrl+C in its terminal, or close the application using that port. Retain the default ports for this guide and the browser tests.

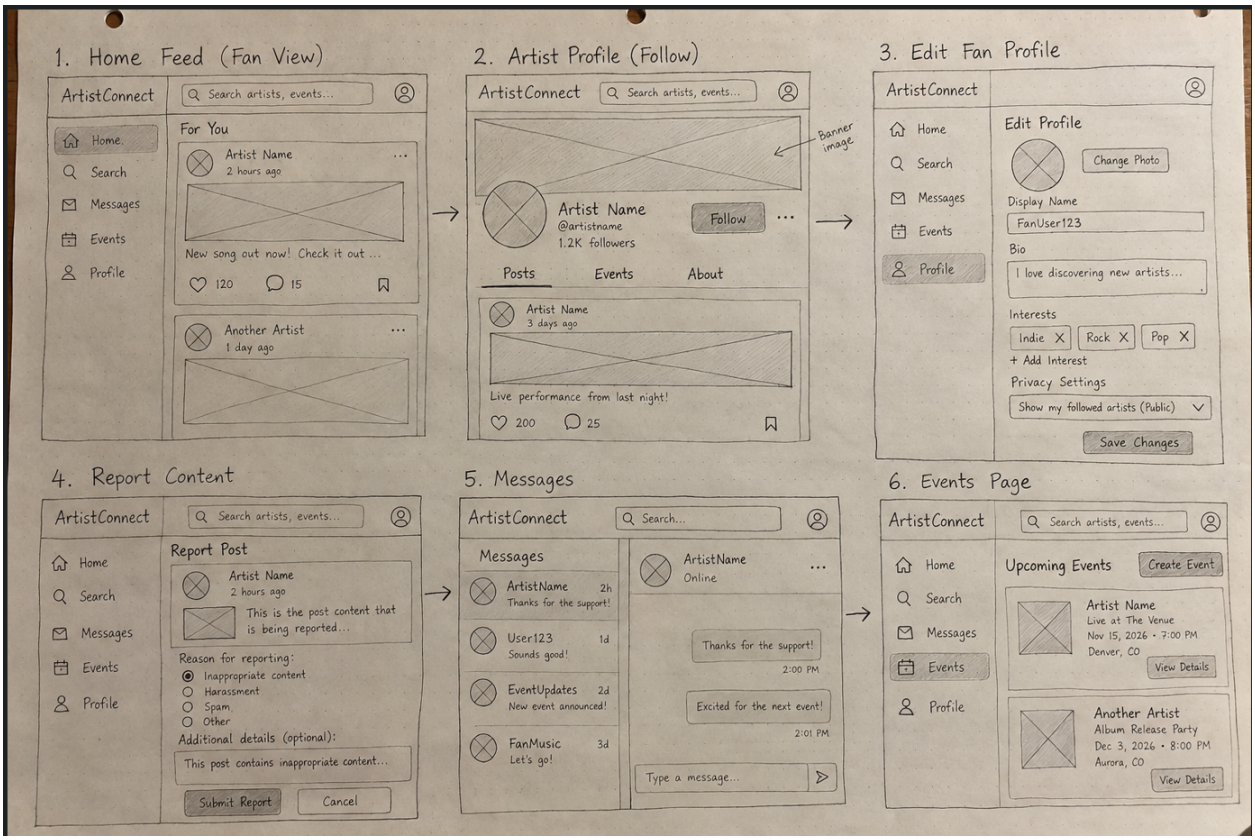
The interface opens but live features fail.
Confirm that the backend remains running and that its artist-list endpoint returns JSON. Open the frontend through `http://localhost:3000`, not a different host name or port.

Playwright cannot find its module or browser.
Repeat the optional Playwright installation. Then, from `frontend`, run:

```
npx playwright install chromium
```

Demo data reset or browser tests fail.
The default H2 database is in memory with schema recreation enabled. Restarting the backend, including a DevTools restart, restores its seeded records. Browser reloads alone do not reset it. Restart the backend before a presentation or seed-dependent test. Some existing demo features store data in browser local storage; restarting the backend does not clear that separate browser data.

16. UI Mockups



16.1 Make a Post – Artist

Figure 16.1 shows the artist's post creation interface, including the story field, media upload control, visibility settings, comment preferences, and a live preview.

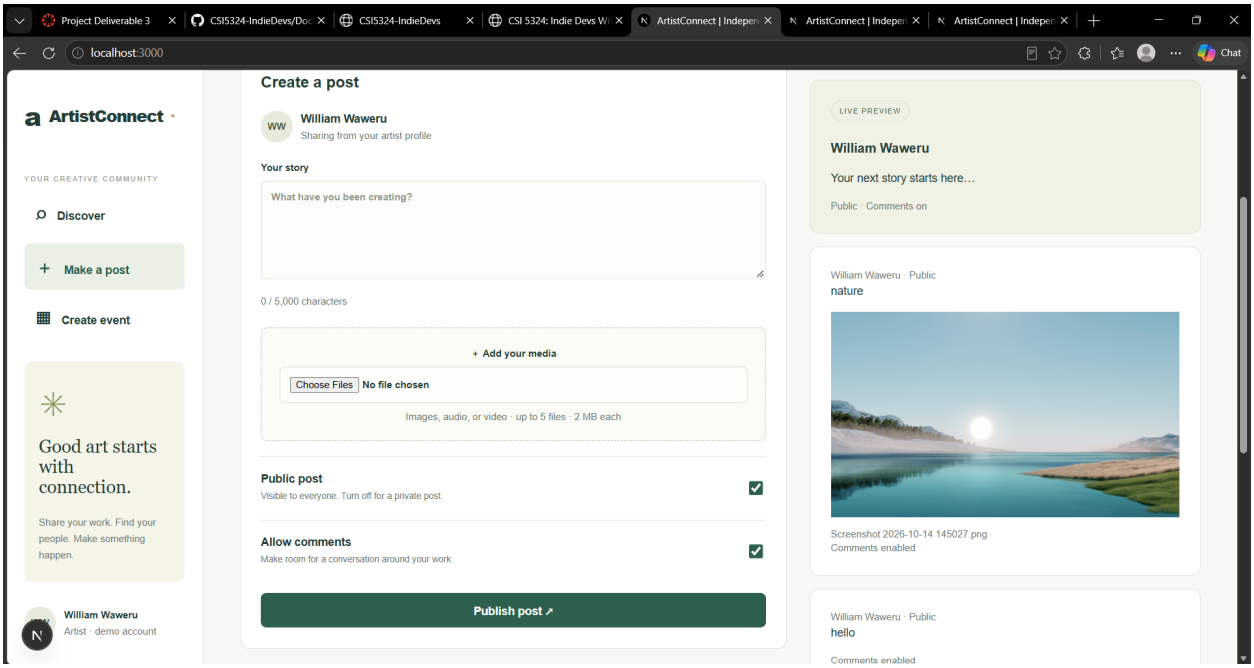


Figure 16.1: ArtistConnect UI preview for the Make a Post use case.

16.2 Search for Artist – Fan

Figure 16.2 shows the artist discovery interface, where a fan can search using artist name, genre or style, and location, then view matching artist profiles.

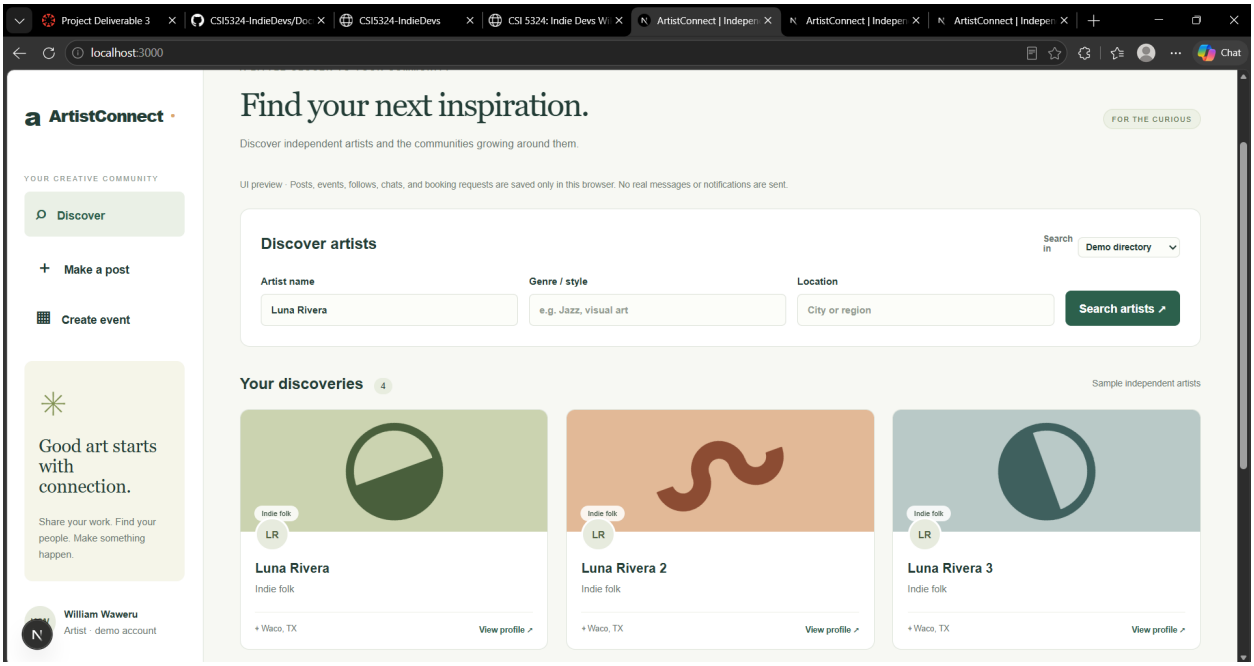


Figure 16.2: ArtistConnect UI preview for the Search for Artist use case.

16.3 Create Event – Artist

Figure 16.3 shows the event creation interface, including event details, venue, capacity, description, ticket tiers, and draft or review submission actions.

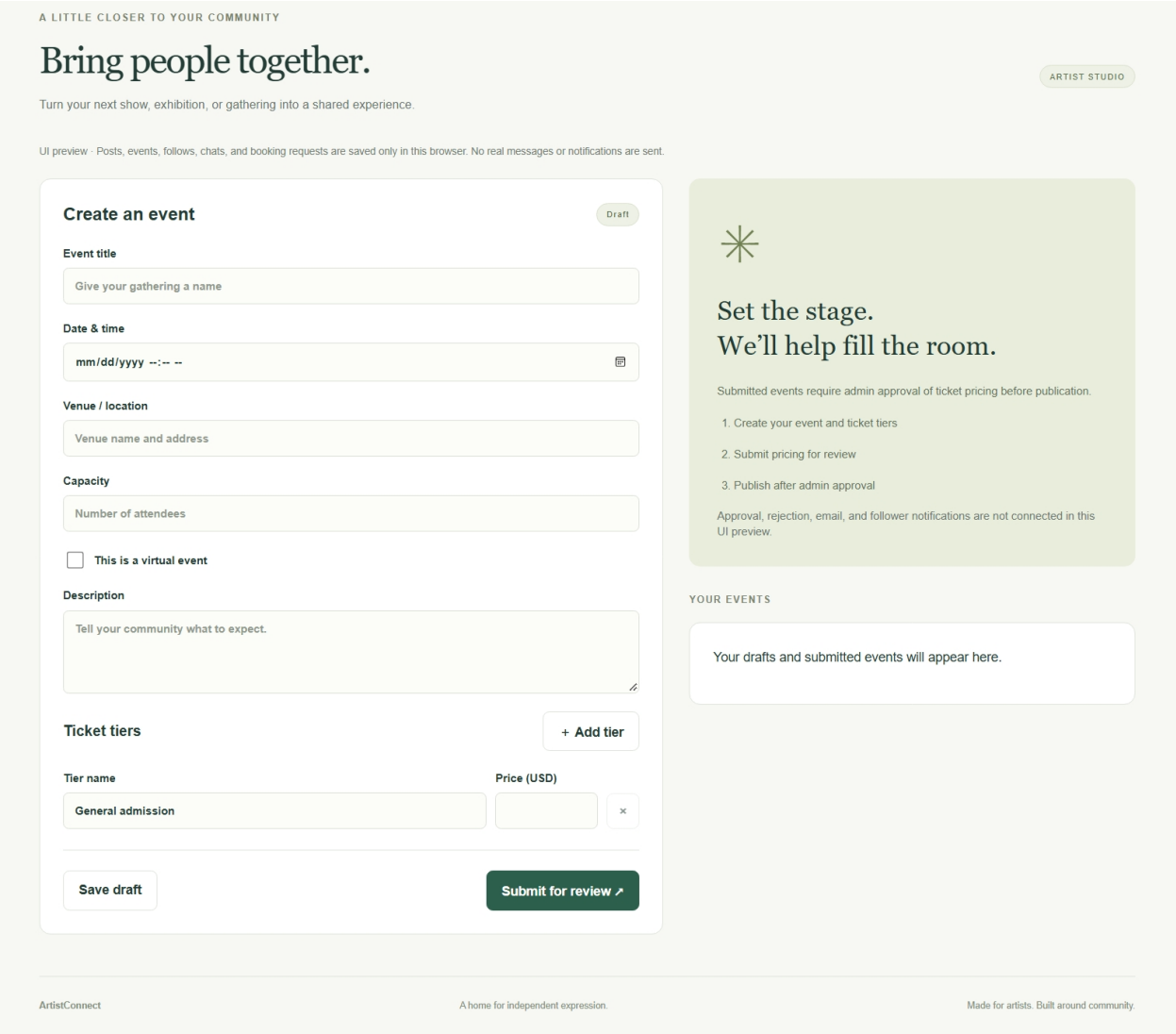


Figure 16.3: ArtistConnect UI preview for the Create Event use case.